

A1: Data Wrangling I

Data Wrangling Practical (Titanic Dataset)

LINE-BY-LINE CODE EXPLANATION

◆ BLOCK 1 — Import Libraries

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
```

Line	What it does	Why we need it
<code>import pandas as pd</code>	Is the Pandas library, nicknamed <code>pd</code>	Load, clean, and manipulate tabular data (like Excel)
<code>import numpy as np</code>	Is NumPy, nicknamed <code>np</code>	Numerical operations, arrays, and functions
<code>from sklearn.preprocessing import MinMaxScaler</code>	Imports the Min-Max scaling utility	Normalize numerical columns to range

To examiner: These are industry-standard libraries. Pandas handles DataFrames, NumPy handles numerical computation, and Scikit-learn provides preprocessing utilities like normalization.

◆ BLOCK 2 — Load Dataset

```
df = pd.read_csv("Titanic-Dataset.csv")
print("=== First 5 Rows ===")
print(df.head())
```

Line	What it does
<code>pd.read_csv("Titanic-Dataset.csv")</code>	Loads the CSV file from disk into a DataFrame called <code>df</code>
<code>df.head()</code>	Shows the first 5 rows to visually verify data loaded correctly

EXAM TIP — If filename is different:

- Look at the file on desktop/folder
- Just change the filename: `pd.read_csv("whatever_name.csv")`
- If it's in a folder: `pd.read_csv("folder/filename.csv")`
- To check current directory in Jupyter: `import os` then `os.listdir()`

◆ BLOCK 3 — Data Preprocessing

```
print(df.isnull().sum())
print(df.describe())
```

```
print(df.dtypes)
print(df.shape)
```

Line by line:

```
df.isnull().sum()
```

- `isnull()` → marks every missing cell as **True**
- `.sum()` → counts how many **True** values per column
- **Why:** Tells us exactly which columns have missing data and how many

```
df.describe()
```

- Gives **count, mean, std, min, 25%, 50%, 75%, max** for all numeric columns
- **Why:** Quick statistical summary — helps spot outliers, wrong values, weird ranges

```
df.dtypes
```

- Shows data type of every column (int64, float64, object)
- **Why:** We need to know which columns are numbers and which are text before processing

```
df.shape
```

- Returns (**rows, columns**) — e.g., (891, 12)
- **Why:** Confirms dataset dimensions before and after cleaning

◆ BLOCK 4 — Data Formatting & Normalization

```
df["Age"] = df["Age"].fillna(df["Age"].median())
```

```
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
```

Why median for Age?

- Age has outliers (very old/very young passengers)
- Median is not affected by outliers — it's the **middle value**
- Mean would get pulled by extreme values

Why mode for Embarked?

- Embarked is **categorical** (C, Q, S) — you can't take mean/median of letters
- Mode = most frequently occurring value → most logical fill

```
df.drop(columns=["Cabin", "Name", "Ticket", "PassengerId"], inplace=True)
```

Column dropped

Why

in

% values missing — too much to recover

Column dropped

Why

<code>sex</code>	a label — no analytical value
<code>cabin</code>	long alphanumeric — no pattern
<code>passengerId</code>	a serial number — meaningless for analysis

```
scaler = MinMaxScaler()
df[["Age", "Fare"]] = scaler.fit_transform(df[["Age", "Fare"]])
```

Min-Max Normalization Formula:

$$X_{\text{new}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

- Scales all values to **range [0, 1]**
- `fit_transform` = learns min/max from data AND transforms it in one step
- **Why Age and Fare?** These have very different scales (Age: 0–80, Fare: 0–500). ML models get confused by huge differences in scale.

◆ BLOCK 5 — Categorical to Quantitative

```
df["Sex"] = df["Sex"].map({"male": 0, "female": 1})
```

- **Binary Encoding** — only 2 categories, so map directly to 0 and 1
- Machine learning models need numbers, not text

```
df = pd.get_dummies(df, columns=["Embarked"], drop_first=True)
```

- **One-Hot Encoding** — Embarked has 3 values: C, Q, S
- Creates new binary columns: `Embarked_Q`, `Embarked_S`
- `drop_first=True` → drops `Embarked_C` to avoid **dummy variable trap** (multicollinearity)
- Result: C = [0,0], Q = [1,0], S = [0,1]

EXAM ADAPTABILITY GUIDE

— If dataset name is unknown, do this first —

```
import os
print(os.listdir())      # See all files in current directory
print(os.listdir("data/")) # If inside a folder
```

— Load accordingly —

```
df = pd.read_csv("whatever_file_you_see.csv")
```

— If different columns have missing values —

For numeric columns:

```
df["column_name"] = df["column_name"].fillna(df["column_name"].median())
```

For categorical columns:

```
df["column_name"] = df["column_name"].fillna(df["column_name"].mode()[0])
```

```
# — If different columns need dropping —  
# Drop columns with >40% missing or irrelevant ones  
df.drop(columns=["col1", "col2"], inplace=True)  
  
# — Always check what columns exist first —  
print(df.columns)  
print(df.isnull().sum())  
print(df.dtypes)
```



PART 3: ALL POSSIBLE VIVA QUESTIONS & ANSWERS

Q1. What is Data Wrangling?

Data Wrangling (also called Data Munging) is the process of **cleaning, structuring, and enriching raw data** into a desired format for better analysis. It includes handling missing values, removing duplicates, type conversions, normalization, and encoding.

Q2. What is a DataFrame?

A DataFrame is a **2-dimensional labeled data structure** in Pandas, similar to an Excel spreadsheet or SQL table. It has rows and columns, where each column can hold a different data type.

Q3. What does `isnull().sum()` do?

`isnull()` returns a boolean DataFrame where **True** indicates a missing value. `.sum()` counts the **True** values per column, giving the **total number of missing values in each column**.

Q4. What does `df.describe()` show?

It shows **descriptive statistics** for all numeric columns:

- count, mean, std (standard deviation)
- min, 25th percentile, median (50%), 75th percentile, max

Q5. What is `df.shape`?

Returns a tuple (`rows`, `columns`) — the **dimensions of the DataFrame**.

Q6. What is `df.dtypes`?

Shows the **data type of each column** — int64, float64, or object (text/categorical).

Q7. What does `df.head()` do?

Displays the **first 5 rows** of the DataFrame. You can pass a number like `df.head(10)` to see more.

Q8. Why do we use median to fill missing Age values instead of mean?

Age data is often **skewed** with outliers (very old/very young). The **median is robust to outliers** — it's simply the middle value when sorted. Mean gets distorted by extreme values. For example, if most passengers are 20–40 but a few are 80+, the mean will be pulled high, but the median stays accurate.

Q9. Why do we use mode to fill missing Embarked values?

Embarked is a **categorical variable** (C, Q, S). We cannot compute mean or median for categories. Mode is the **most frequently occurring value**, making it the most logical and statistically sound choice for filling categorical missing data.

Q10. What is Min-Max Normalization? Give the formula.

Min-Max Normalization scales data to a fixed range, usually **[0, 1]**.

Formula: $X_{\text{new}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$

It ensures no single feature dominates due to its scale. For example, Fare (0–500) would dominate Age (0–80) without normalization.

Q11. What is the difference between normalization and standardization?

	Normalization (Min-Max)	Standardization (Z-score)
range	fixed range [0, 1]	fixed range
formula	$(x - \min) / (\max - \min)$	$(x - \text{mean}) / \text{std}$
when	outliers, bounded range needed	outliers present, Gaussian distribution
python	<code>MinMaxScaler</code>	<code>StandardScaler</code>

Q12. What is One-Hot Encoding? Why use `drop_first=True`?

One-Hot Encoding converts a categorical column with N categories into **N binary columns**. `drop_first=True` removes one column to avoid the **dummy variable trap** — where one column is perfectly predictable from the others (multicollinearity), which breaks many ML models.

Q13. What is Binary Encoding and when is it used?

Binary Encoding maps a categorical variable with **exactly 2 categories** directly to 0 and 1. Example: male=0, female=1. It's simpler than One-Hot when only 2 values exist.

Q14. Why did we drop the Cabin column?

The Cabin column has approximately **77% missing values**. Imputing that much missing data would introduce too much artificial information and bias the analysis. It's safer to drop it entirely.

Q15. What is `fit_transform()` in `MinMaxScaler`?

- `fit()` → **learns** the min and max values from the data
- `transform()` → **applies** the scaling formula

- `fit_transform()` → does **both in one step**

Important: On test/new data, we only use `transform()` (not `fit_transform()`) so it uses the same min/max learned from training data.

Q16. What are all the methods of Data Wrangling?

1. **Data Cleaning** — fix/remove missing, duplicate, or incorrect data
2. **Data Integration** — combining data from multiple sources
3. **Data Reduction** — reducing volume while maintaining integrity (PCA, sampling)
4. **Data Transformation** — normalizing, aggregating, encoding
5. **Data Discretization** — converting continuous data to discrete bins/intervals

Q17. What are the 5 V's of Big Data?

1. **Volume** — Huge amount of data (TBs, PBs)
2. **Velocity** — Speed at which data is generated (real-time streams)
3. **Variety** — Different types: structured, semi-structured, unstructured
4. **Veracity** — Trustworthiness/accuracy of data
5. **Value** — Usefulness of data for decision making

Q18. What is the Data Science Life Cycle?

1. **Discovery** — Understand the problem
2. **Data Preparation** — Collect, clean, wrangle data
3. **Model Planning** — Choose algorithms, features
4. **Model Building** — Train and test models
5. **Communicate Results** — Visualize, report findings
6. **Operationalize** — Deploy model into production

Q19. What is the difference between Data Cleaning and Data Transformation?

- **Data Cleaning:** Fixing errors — handling missing values, removing duplicates, correcting data types
- **Data Transformation:** Changing data format/scale — normalization, encoding, aggregation, log transforms

Q20. What is multicollinearity and how does `drop_first=True` prevent it?

Multicollinearity is when two or more predictor variables are **highly correlated**, making it hard for models to determine individual effects. In One-Hot Encoding with 3 categories (C, Q, S), if Embarked_Q=0 and Embarked_S=0, we **already know** it's C — so Embarked_C is redundant. Dropping it removes this perfect correlation.

Q21. What are different ways to handle missing data?

1. **Drop rows** — `df.dropna()` — when very few rows have missing values
2. **Drop columns** — when column has >40-50% missing
3. **Fill with mean** — for normally distributed numeric data
4. **Fill with median** — for skewed numeric data

5. **Fill with mode** — for categorical data
6. **Forward/Backward fill** — for time-series data:
`df.fillna(method='ffill')`
7. **Interpolation** — estimate from neighboring values
8. **ML-based imputation** — predict missing values using other features

Q22. What is the difference between `map()` and `get_dummies()`?

- `map()` → used for **simple substitution** of values in a single column (e.g., male→0, female→1). Works with a dictionary.
- `get_dummies()` → used for **One-Hot Encoding** of categorical columns — creates multiple new binary columns automatically.

Q23. What is Label Encoding vs One-Hot Encoding? When to use which?

	Label Encoding	One-Hot Encoding
how	maps integer to each category (C=0, Q=1, S=2)	creates binary column for each category
problem	loses ordering (S > Q > C which is wrong)	loses ordering
when to use	ordinal data (Low/Med/High)	nominal data (no order)
python	<code>LabelEncoder</code>	<code>get_dummies</code> or <code>OneHotEncoder</code>

Q24. What happens if you apply `fit_transform()` on test data?

It would **re-learn new min/max** from test data, causing **data leakage** — the model would scale test data differently from training data, giving incorrect and inconsistent results. Always use `fit()` on training data, and `transform()` only on test data.

Q25. What is the difference between structured, semi-structured, and unstructured data?

- **Structured** — Organized in rows/columns, e.g., CSV, SQL tables (like Titanic dataset)
- **Semi-structured** — Has some organization but not fully tabular, e.g., JSON, XML
- **Unstructured** — No fixed format, e.g., images, videos, audio, social media text

How to Present to the Examiner — Question by Question

Here's exactly what to **say and point to** on screen:

Q1 — Import Libraries

"I have imported pandas for data manipulation, numpy for numerical operations, and MinMaxScaler from sklearn for normalization."

Q2 — Dataset

"I am using the Titanic dataset, which is an open-source dataset available at this GitHub URL. It contains information about 891 passengers — including their survival status, age, fare, sex, and port of embarkation."

Q3 — Load Dataset

"I loaded the dataset directly from the URL using `pd.read_csv()` into a pandas DataFrame. Here you can see the first 5 rows using `df.head()`."

Q4 — Preprocessing (point to each output)

"Using `isnull().sum()`, I can see Age has 177 missing values, Cabin has 687, and Embarked has 2. I used `describe()` to get statistical summary — count, mean, standard deviation, min, max. Using `dtypes`, I identified variable types — integer, float, and object. The dimensions are 891 rows and 12 columns, confirmed by `df.shape`."

Q5 — Formatting & Normalization

"I filled missing Age values with the median, and missing Embarked values with the mode, since these are the most appropriate strategies. I dropped Cabin, Name, Ticket, and PassengerId as they are not useful for analysis. Then I applied Min-Max Normalization on Age and Fare to scale them between 0 and 1, so no single feature dominates due to its range."

Q6 — Categorical to Quantitative

"For the Sex column, I used Binary Encoding — male becomes 0 and female becomes 1. For the Embarked column which has 3 categories (C, Q, S), I used One-Hot Encoding using `pd.get_dummies()` with `drop_first=True` to avoid the dummy variable trap. The final DataFrame has 9 columns, all in numeric format, ready for machine learning."

One golden tip 🏆 — when the examiner asks "why did you choose median for Age?", say: "Median is preferred over mean for filling missing values because it is not affected by outliers, and Age can have extreme values."

A2: Data Wrangling II

📌 **Practical Title:** Group A — Data Wrangling II | Academic Performance Dataset
(Manually Created)

📌 **Problem Statement:** Create a student Academic Performance dataset and perform: (1) Scan and fix missing values and inconsistencies, (2) Detect and handle outliers in numeric variables, (3) Apply data transformation on a skewed variable to normalize its distribution.

Data Wrangling II

📋 PART 1: LINE-BY-LINE CODE EXPLANATION

◆ BLOCK 1 — Import Libraries

```
import pandas as pd
import numpy as np
from scipy import stats
import warnings
warnings.filterwarnings("ignore")
```

Line	What it does	Why
<code>pd as pd</code>	Frame operations	df, clean, manipulate tabular data
<code>np as np</code>	numerical operations	random data generation, math, Numpy
<code>from scipy import stats</code>	statistical functions	mean, skewness tests (kept for reference)
<code>warnings.filterwarnings("ignore")</code>	suppresses warning messages output during execution	

◆ BLOCK 2 — Create Dataset Manually

```
np.random.seed(42)
n = 50
```

- `random.seed(42)` → fixes randomness so **same data generates every run**
- `n = 50` → we are creating 50 student records

```
data = {
    "StudentID" : range(1, n+1),
    "Age"       : np.random.randint(17, 23, n).astype(float),
    "Gender"    : np.random.choice(["Male", "Female", None], n, p=[0.48, 0.48, 0.04]),
    "Attendance": np.random.uniform(55, 100, n).round(1),
    "MathScore" : np.random.randint(40, 100, n).astype(float),
    "SciScore"  : np.random.randint(40, 100, n).astype(float),
    "StudyHrs"  : np.random.uniform(0.5, 8, n).round(1),
}
```

Column	Generation Method	Why that method
StudentID	<code>randint(1, 51)</code>	Unique serial numbers
Age	<code>randint(17, 23)</code>	Realistic ages, cast to float for NaN support
Gender	<code>rand.choice</code> with <code>p=</code>	Probability: 48% Male, 48% Female, 4% None (missing)
Attendance	<code>round(rand(55, 100))</code>	Percentage, rounded to 1 decimal
MathScore	<code>randint(40, 100)</code>	Integer scores
SciScore	<code>randint(40, 100)</code>	Integer scores
StudyHrs	<code>round(rand(0.5, 8))</code>	Percentage hours per day

◆ BLOCK 3 — Inject Problems Intentionally

Missing values

```
df.loc[[3, 10, 22], "Age"] = np.nan
df.loc[[5, 15, 30], "MathScore"] = np.nan
df.loc[[7, 20], "Attendance"] = np.nan
```

Inconsistencies

```
df.loc[0, "MathScore"] = 150 # impossible: score > 100
df.loc[1, "Attendance"] = -10 # impossible: negative attendance
```

Outliers

```
df.loc[2, "StudyHrs"] = 22 # outlier: no one studies 22 hrs/day
df.loc[11, "SciScore"] = 4 # outlier: extremely low score
```

Why inject problems? Because real-world data always has these issues. We are **simulating real-world dirty data** so we can practice cleaning it. This is the whole point of the practical.

◆ BLOCK 4 — Q1: Scan & Fix Missing Values + Inconsistencies

```
print(df.isnull().sum())
```

- Counts missing values per column — tells us **where** and **how many**

```
print("MathScore > 100 :", df[df["MathScore"] > 100][["StudentID", "MathScore"]])
print("Attendance < 0 :", df[df["Attendance"] < 0][["StudentID", "Attendance"]])
```

- **Boolean filtering** to find rows where values are **logically impossible**
- Score > 100 is invalid, attendance < 0 is impossible

```
df.loc[df["MathScore"] > 100, "MathScore"] = np.nan
df.loc[df["Attendance"] < 0, "Attendance"] = np.nan
```

- Replace impossible values with **NaN** — now they become **missing values** and get handled in the next step cleanly

```
df["Age"] = df["Age"].fillna(df["Age"].median())
df["MathScore"] = df["MathScore"].fillna(df["MathScore"].mean()).round(1)
df["Attendance"] = df["Attendance"].fillna(df["Attendance"].median())
df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])
```

Column	Fill Method	Why
	an	eric, may have slight skew
MathScore	mean	eric, approximately normal distribution
Attendance	median	eric, robust to outliers
Gender	mode	logical — only mean/median makes no sense

◆ BLOCK 5 — Q2: Outlier Detection using IQR Method

```
Q1 = df[col].quantile(0.25)
Q3 = df[col].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
```

IQR Method — The Formula:

```
IQR = Q3 - Q1
Lower Fence = Q1 - 1.5 × IQR
Upper Fence = Q3 + 1.5 × IQR
Any value outside these fences = OUTLIER
```

- **Q1** = 25th percentile (bottom quarter)
- **Q3** = 75th percentile (top quarter)
- **IQR** = middle 50% spread of data
- **1.5 × IQR** = standard multiplier (Tukey's rule)

```
outliers = df[(df[col] < lower) | (df[col] > upper)][col]
```

- Finds all values **below lower fence OR above upper fence**

Handling Outliers — Capping (Winsorization):

```
df[col] = df[col].clip(lower=lower, upper=upper)
```

- **.clip()** → any value **below lower** becomes exactly **lower**, any value **above upper** becomes exactly **upper**
- We don't delete the row — we just **cap** it at the boundary

- This preserves data size while removing extreme influence

◆ BLOCK 6 — Q3: Data Transformation (Log Transform)

```
print("StudyHrs skewness:", round(df["StudyHrs"].skew(), 4))
```

- `.skew()` → measures asymmetry of distribution
- Positive skew = tail on right side = a few very high values pulling the mean up

```
df["StudyHrs_log"] = np.log1p(df["StudyHrs"])
```

- `log1p(x) = log(1 + x)` — **safe version of log**
- Why `log1p` not `log`? Because `log(0) = undefined/infinity` — `log1p` handles zero values safely
- **Effect:** Compresses large values, stretches small values → makes distribution more symmetric/normal

Before: StudyHrs → skewed (outlier at 22 pulled it)

After: `log(1+x)` → skewness reduced, closer to normal bell curve

🎓 PART 2: ALL POSSIBLE VIVA QUESTIONS & ANSWERS

● EASY LEVEL

Q1. What is Data Wrangling?

Data Wrangling is the process of **cleaning, structuring, and enriching raw data** into a usable format. It covers handling missing values, fixing inconsistencies, removing outliers, and transforming variables.

Q2. Why did you create the dataset manually instead of using a CSV?

The problem statement says to **create** an academic performance dataset. Manual creation using NumPy allows us to control the data, intentionally inject missing values, inconsistencies, and outliers to practice all cleaning techniques.

Q3. What does `isnull().sum()` tell us?

It tells us the **count of missing (NaN) values in each column**, helping us decide which columns need imputation.

Q4. What is `np.random.seed(42)` and why is it used?

It sets a **fixed starting point for random number generation**. Using `seed(42)` ensures the same random data is generated every time the code runs — making results **reproducible**.

Q5. What is `np.nan`?

`np.nan` stands for **Not a Number** — it is NumPy's representation of a missing or undefined value. Pandas uses it to mark missing data.

● MEDIUM LEVEL

Q6. What is an inconsistency in data? Give examples from this practical.

An inconsistency is a value that is **logically or domain-wise impossible**:

- `MathScore = 150` → scores can only be 0–100
- `Attendance = -10` → percentage cannot be negative These need to be detected using domain knowledge and fixed before analysis.

Q7. What is an outlier? How did you detect it here?

An outlier is a data point that lies **far from the rest of the data**. We used the **IQR method**:

- Calculate Q1, Q3, IQR = Q3 - Q1
- Lower fence = Q1 - 1.5×IQR, Upper fence = Q3 + 1.5×IQR
- Any value outside these fences is an outlier Example: StudyHrs = 22 was far above the upper fence.

Q8. What is Winsorization / Capping? Why use it over deletion?

Capping (Winsorization) **replaces outlier values with the fence boundary values** instead of deleting the row.

- `clip(lower, upper)` sets values below lower → lower, above upper → upper
- **Advantage over deletion**: Preserves dataset size, doesn't lose entire rows, reduces outlier impact while retaining the record.

Q9. Why use `log1p` instead of `log` for transformation?

`log(0)` is mathematically **undefined** ($-\infty$). If any value in the column is 0, `np.log()` will produce `-inf` or error. `np.log1p(x)` computes `log(1+x)`, which is **safe for zero and near-zero values**.

Q10. What is skewness? What does positive vs negative skewness mean?

Skewness measures the **asymmetry of a distribution**:

- **Positive skew (right-skewed)**: Long tail on the right, mean > median. A few very large values.
- **Negative skew (left-skewed)**: Long tail on the left, mean < median. A few very small values.
- **Zero skew**: Perfectly symmetric (normal distribution) Log transformation reduces positive skewness.

Q11. Why did you fill MathScore with mean but Age with median?

- **MathScore**: Scores tend to be approximately normally distributed → mean is appropriate
- **Age**: Can have slight outliers (much older students) → median is more robust General rule: Use **mean for symmetric data, median for skewed or outlier-prone data**.

Q12. What is `.astype(float)` and why was it needed for Age?

Pandas integer columns **cannot store NaN values** (in older pandas versions). Converting to `float` allows the column to hold `NaN`, which is represented as a floating-point special value.

● HARD / ADVANCED LEVEL

Q13. What are all the methods to handle outliers?

1. **Deletion** — Remove the outlier rows entirely (use when few outliers, large dataset)
2. **Capping/Winsorization** — Replace with fence values using `.clip()`
3. **Mean/Median Imputation** — Replace outlier with central tendency value
4. **Log/Sqrt Transformation** — Reduce the effect of extreme values
5. **Z-score method** — Remove rows where $|Z| > 3$ (beyond 3 standard deviations)
6. **IQR method** — Most common, non-parametric, doesn't assume normal distribution

Q14. What is the difference between IQR method and Z-score method for outlier detection?

	IQR Method	Z-Score Method
Imputation	distribution assumption	times normal distribution
Formula	$1.5 \times \text{IQR}$, $Q3 + 1.5 \times \text{IQR}$	3 ($Z = (x - \text{mean}) / \text{std}$)
Use for	skewed data	normally distributed data
Robustness	more robust	sensitive to extreme outliers

Q15. What are all the Data Transformation techniques?

1. **Log Transform** — `log(x)` or `log1p(x)` → reduces right skew
2. **Square Root Transform** — `sqrt(x)` → moderate skew reduction
3. **Box-Cox Transform** — general power transform, finds best lambda
4. **Z-score Standardization** — $(x - \text{mean}) / \text{std}$ → mean=0, std=1
5. **Min-Max Normalization** — $(x - \text{min}) / (\text{max} - \text{min})$ → range [0, 1]
6. **Binning/Discretization** — convert continuous to categorical bins
7. **Power Transform** — x^2 or x^3 for specific relationships

Q16. What are all the methods of handling missing data?

1. **Listwise Deletion** — `df.dropna()` → remove rows with any missing value
2. **Column Deletion** — drop column if >40-50% missing
3. **Mean Imputation** — fill with column mean (symmetric numeric)
4. **Median Imputation** — fill with column median (skewed numeric)
5. **Mode Imputation** — fill with most frequent value (categorical)
6. **Forward Fill** — `fillna(method='ffill')` → use previous row value (time-series)

7. **Backward Fill** — `fillna(method='bfill')` → use next row value
8. **Interpolation** — estimate from neighboring values
9. **KNN Imputation** — use K nearest neighbors to predict missing value
10. **Model-based** — train ML model to predict missing values

Q17. What are the 5 main steps/methods of Data Wrangling?

1. **Data Cleaning** — Handle missing values, duplicates, inconsistencies
2. **Data Integration** — Merge data from multiple sources (merge, join, concat)
3. **Data Reduction** — Reduce size: PCA, sampling, feature selection
4. **Data Transformation** — Normalize, encode, log transform
5. **Data Discretization** — Convert continuous to bins (e.g., Age → Young/Middle/Senior)

Q18. What is the purpose of data transformation? Give 3 reasons.

1. **Change scale** — Normalization brings all features to same range for ML algorithms
2. **Convert non-linear to linear** — Log transform can linearize exponential relationships
3. **Reduce skewness** — Log/sqrt transform makes skewed data approach normal distribution, which many statistical tests require

Q19. What is the difference between Data Cleaning and Data Transformation?

- **Data Cleaning:** Fixes **errors and quality issues** — missing values, duplicates, inconsistencies, outliers
- **Data Transformation:** Changes **format or scale** of valid data — normalization, encoding, log transform, binning

Q20. Why is normal distribution important in data science?

Many statistical tests (t-test, ANOVA, linear regression) and ML algorithms **assume normally distributed data**. If data is skewed, these methods give incorrect results. Transformations like log or Box-Cox help convert skewed data to near-normal so these techniques can be validly applied.

 QUICK REVISION CHEATSHEET

`random.seed(42)` → reproducible random data
`isnull().sum()` → count missing per column
`df[df["col"] > val]` → filter inconsistencies
`fillna(median())` → fill numeric skewed missing
`fillna(mean())` → fill numeric normal missing
`fillna(mode()[0])` → fill categorical missing
`Q1, Q3, IQR` → quartile-based outlier detection
`clip(lower, upper)` → cap/winsorize outliers
`df["col"].skew()` → measure skewness
`np.log1p(x)` → log transform safe for zeros
`log1p` reduces → positive/right skewness

 What the Code Does — Full Explanation

Dataset Created

50 student records with columns: `StudentID`, `Age`, `Gender`, `Attendance`, `MathScore`, `SciScore`, `StudyHrs` — with **deliberately injected** missing values, inconsistencies, and outliers so all operations are visible clearly.

Q1 — Missing Values & Inconsistencies

Two problems handled:

Problem	Column	Fix Applied
Missing values	<code>MathScore</code> , <code>Attendance</code> , <code>Gender</code>	Filled with median/mean/mode
Impossible values	<code>MathScore=150</code> , <code>Attendance=-10</code>	Replaced with NaN first, then filled

Why median/mean/mode?

- `Age` & `Attendance` → **median** (robust to outliers)
 - `MathScore` → **mean** (scores are normally distributed)
 - `Gender` → **mode** (categorical column)
-

Q2 — Outliers Detection & Handling

Method used: IQR (Interquartile Range)

Formula:

Lower Bound = $Q1 - 1.5 \times IQR$

Upper Bound = $Q3 + 1.5 \times IQR$

Any value outside this range = outlier.

Handling method: Capping (Winsorization) Using `df[col].clip(lower, upper)` — outliers are capped to the boundary value instead of being deleted, so no data is lost.

Q3 — Data Transformation

Column chosen: `StudyHrs` (right-skewed — most students study few hours, a few study very long)

Transformation: Log Transformation using `np.log1p()`

- Converts right skew → near-normal distribution
 - Makes the variable more suitable for statistical analysis and ML models
 - `log1p` is used instead of `log` to safely handle 0 values
-

How to Present to the Examiner

Q1:

"I created a dataset with 50 students. I used `isnull().sum()` to detect missing values in Age, MathScore, Attendance, and Gender. I also found inconsistencies — MathScore of 150 and Attendance of -10 which are logically impossible. I converted those to NaN first, then filled missing values using median for numeric columns and mode for the categorical Gender column."

Q2:

"I used the IQR method to detect outliers across all numeric columns. The IQR method calculates Q1 and Q3, and any value beyond 1.5 times the IQR from these quartiles is an outlier. I handled them using Capping — clipping values to the boundary — so no rows are deleted and data size is preserved."

Q3:

"StudyHrs was right-skewed, meaning most students study less and very few study a lot. I applied Log Transformation using `np.log1p()` to reduce this skewness and bring the distribution closer to normal. After transformation, the skewness value decreases significantly, making it better for analysis and modeling."

 **Bonus answer if examiner asks "why not delete outliers?"**

"Deleting outliers reduces our dataset size. Since the dataset is small (50 rows), capping is better — it keeps all rows while limiting the extreme effect of outliers."

A3: Descriptive Statistics

📌 **Practical Title:** Group A — Descriptive Statistics | Measures of Central Tendency & Variability | Employee/Student Dataset + Iris Dataset

📌 **Problem Statement:** (1) Provide summary statistics (mean, median, min, max, std) of numeric variables grouped by a categorical variable. Create a numeric list per category. (2) Display statistical details (percentile, mean, std, etc.) for each species in the Iris dataset.

Descriptive Statistics

📋 PART 1: LINE-BY-LINE CODE EXPLANATION

◆ BLOCK 1 — PART 1: Load CSV Dataset

```
import pandas as pd
```

```
df = pd.read_csv("data.csv")
print(df.head())
print(df.shape)
print(df.columns)
```

Line	What it does	Why
<code>read_csv("data.csv")</code>	Is CSV into DataFrame	Load external dataset
<code>head()</code>	Shows first 5 rows	Initial verification of data
<code>shape</code>	Returns (rows, cols)	Check dataset dimensions
<code>columns</code>	Lists all column names	Verify what variables exist

⚠️ **EXAM TIP:** If file is named differently, do `import os;`
`print(os.listdir())` first to find correct filename.

◆ BLOCK 2 — Grouped Summary Statistics

```
grouped = df.groupby("Grade_Group")["Score"].agg(
    Mean = "mean",
    Median = "median",
    Min = "min",
    Max = "max",
    Std = "std"
).round(2)
print(grouped)
```

Step by step:

```
df.groupby("Grade_Group")
```

- **Splits** the DataFrame into separate groups based on unique values of `Grade_Group`

- Like sorting students into piles by their grade (A, B, C, D...)

["Score"]

- From each group, **select only the Score column**
- We want statistics on Score, not all columns

.agg(Mean="mean", Median="median", Min="min", Max="max", Std="std")

- `.agg()` = **aggregate** — applies multiple functions at once
- Each argument: `OutputColumnName = "function_name"`
- Computes all 5 statistics in one single line

.round(2)

- Round all results to 2 decimal places for clean output

What this tells us: For each grade group, we get a complete statistical picture of scores — helping us compare performance across groups.

◆ BLOCK 3 — Numeric List per Category

```
grade_list = {
    grade: group["Score"].tolist()
    for grade, group in df.groupby("Grade_Group")
}
for grade, scores in grade_list.items():
    print(f"\nGrade {grade} ({len(scores)} students)")
    print(scores)
```

Line by line:

```
{grade: group["Score"].tolist()
 for grade, group in df.groupby("Grade_Group")}
```

- This is a **dictionary comprehension**
- `df.groupby("Grade_Group")` gives `(grade_name, sub_dataframe)` pairs
- For each group → extract Score column → convert to Python list
- Result: `{"A": [85, 90, 78, ...], "B": [65, 70, ...], ...}`

```
group["Score"].tolist()
```

- `.tolist()` converts pandas Series to a plain **Python list**

```
for grade, scores in grade_list.items():
    print(f"\nGrade {grade} ({len(scores)} students)")
```

- Loops through dictionary, prints each grade's score list

- `len(scores)` = number of students in that grade group

◆ BLOCK 4 — PART 2: Load Iris from Sklearn

from sklearn import datasets

iris = datasets.load_iris()

df = pd.DataFrame(iris.data, columns=iris.feature_names)

df["species"] = pd.Categorical.from_codes(iris.target, iris.target_names)

Line	What it does
<code>datasets.load_iris()</code>	Is built-in Iris dataset object
<code>iris.data</code>	Py array of 150×4 feature values
<code>iris.feature_names</code>	Array of 4 column names
<code>iris.target</code>	Array of 0,1,2 representing species
<code>iris.target_names</code>	Array: ['setosa','versicolor','virginica']
<code>pd.Categorical.from_codes(...)</code>	Maps 0→setosa, 1→versicolor, 2→virginica

Result: Clean DataFrame with 150 rows, 4 numeric feature columns + 1 species column

◆ BLOCK 5 — Statistical Details per Species

for species in df["species"].unique():

subset = df[df["species"] == species].drop(columns="species")

- `df["species"].unique()` → gets ['setosa', 'versicolor', 'virginica']
- Loop runs **3 times**, once per species
- `df[df["species"] == species]` → **filter rows** for that species only
- `.drop(columns="species")` → remove the species text column so only numeric columns remain for statistics

print(subset.describe().round(3))

- `describe()` gives **8 statistics at once**:
 - count, mean, std, min, 25%, 50% (median), 75%, max
- For all 4 numeric columns simultaneously

pct = subset.quantile([0.10, 0.25, 0.50, 0.75, 0.90])

pct.index = ["10%", "25%", "50%", "75%", "90%"]

print(pct.round(3))

- `.quantile(...)` → computes specific percentile values

- `pct.index = [...]` → renames row labels from 0.10 to "10%" for readability
- **What percentile means:** 25th percentile means 25% of values fall below this number

```
print(subset.mean().round(3))
print(subset.std().round(3))
```

- `.mean()` → average of each column
 - `.std()` → standard deviation of each column — measures **spread** of data
-



PART 2: ALL POSSIBLE VIVA QUESTIONS & ANSWERS

Q1. What are Measures of Central Tendency?

They describe the **center/middle of a dataset**:

- **Mean** — arithmetic average: sum/count
- **Median** — middle value when sorted
- **Mode** — most frequently occurring value
- **Mid-range** — $(\text{max} + \text{min}) / 2$

Q2. What are Measures of Dispersion/Variability?

They describe the **spread of data around the center**:

- **Range** — $\text{max} - \text{min}$
- **Variance** — average of squared deviations from mean
- **Standard Deviation** — square root of variance
- **Mean Deviation** — average of absolute deviations from mean

Q3. What does `groupby()` do?

It **splits the DataFrame into groups** based on unique values of a categorical column, allowing us to apply aggregate functions (mean, sum, count, etc.) to each group separately.

Q4. What does `describe()` return?

Returns **8 statistical measures** for each numeric column: count, mean, std, min, 25th percentile, 50th percentile (median), 75th percentile, max.

Q5. What is the Iris dataset?

A classic dataset with **150 flower samples** from 3 species (setosa, versicolor, virginica), each with 4 measurements: sepal length, sepal width, petal length, petal width. Widely used for classification and statistical analysis.

Q6. What is a percentile?

A percentile indicates the value **below which a given percentage of data falls**. For example, the 25th percentile means 25% of all values are below that number.

Q7. What is the difference between mean and median? When to use which?

	Mean	Median
Definition	Average / Count	Middle value when sorted
Sensitive to outliers	Yes	No
When to use	For symmetric metric data, no outliers	For skewed data, outliers present
Example	Average salary in a company	Median house price

Q8. What is Standard Deviation? What does a high vs low SD tell you?

Standard Deviation (SD) measures how **spread out values are from the mean**.

- **Low SD** → values clustered close to mean → consistent data
- **High SD** → values spread far from mean → high variability
- Formula: $SD = \sqrt{\text{sum}((x - \text{mean})^2) / n}$

Q9. What is `.agg()` and why is it useful?

`.agg()` (aggregate) applies **multiple functions at once** to grouped data. Instead of calling `.mean()`, `.std()`, `.min()` separately, one `.agg()` call returns all of them in a clean table format. Much more efficient.

Q10. What is the difference between Variance and Standard Deviation?

- **Variance** = average of squared differences from mean → unit is squared (e.g., cm²)
- **Standard Deviation** = square root of variance → **same unit as original data** (e.g., cm) SD is preferred for interpretation because it's in the original unit.

Q11. What is `quantile()` and how is it different from `describe()`?

- `describe()` gives fixed percentiles: 25%, 50%, 75%
- `quantile([0.10, 0.25, 0.50, 0.75, 0.90])` lets you **choose any percentiles** you want Both use the same underlying calculation, but `quantile()` gives more flexibility.

Q12. Why do we drop the species column before computing statistics?

`describe()`, `mean()`, `std()` work only on **numeric columns**. The species column is a text/categorical column — computing mean of "setosa, versicolor" makes no sense. Dropping it ensures clean numeric computation.

Q13. What is the difference between Population Standard Deviation and Sample Standard Deviation?

- **Population SD:** Divides by **N** (all data points) → `numpy.std(ddof=0)`
- **Sample SD:** Divides by **N-1** (Bessel's correction) → `pandas.std()` uses `ddof=1` by default. Pandas uses sample SD because in practice we almost always work with **samples**, not entire populations.

Q14. What is the Empirical Rule (68-95-99.7 rule)?

For a **normal distribution**:

- **68%** of data falls within ± 1 SD of mean
- **95%** of data falls within ± 2 SD of mean
- **99.7%** of data falls within ± 3 SD of mean. Used to identify outliers: values beyond ± 3 SD are considered unusual.

Q15. What is the difference between Descriptive and Inferential Statistics?

	Descriptive	Inferential
ose	marize existing data	/ conclusions about population from sample
nples	n, median, SD, percentile	t, chi-square, regression, hypothesis testing
stion answered	at does this data look like?"	at can we conclude about the population?"

Q16. What is Mid-range and when is it useful?

Mid-range = (Maximum + Minimum) / 2 It's the **midpoint of the data range**. Simple to compute but highly sensitive to outliers. Rarely used as a primary measure — useful only when the dataset has no extreme outliers.

Q17. What is a dictionary comprehension and how is it used here?

A dictionary comprehension creates a dictionary in one line: `{key: value for key, value in iterable}` Here: `{grade: group["Score"].tolist() for grade, group in df.groupby("Grade_Group")}` Creates a dict where each key = grade name, value = list of scores for that grade. More efficient than writing a for loop with empty dict.

Q18. What is `pd.Categorical.from_codes()` doing exactly?

Iris target is stored as integers (0, 1, 2). `from_codes(iris.target, iris.target_names)` maps:

- 0 → 'setosa', 1 → 'versicolor', 2 → 'virginica' Creates a **Categorical dtype** in pandas, which is memory-efficient for repeated string values.

Q19. Why is statistics important in Data Science?

Statistics is the **mathematical foundation** of data science:

- Descriptive stats summarize and understand data
- Inferential stats help make predictions and decisions
- Statistical concepts underlie all ML algorithms (mean, variance, distributions, probability)
- Without statistics, we cannot validate models, detect patterns, or interpret results

Q20. What is the difference between grouped statistics and overall statistics?

- **Overall statistics** give one value for the entire column (e.g., mean score of all students)
- **Grouped statistics** give values per category (e.g., mean score per grade group) Grouped stats reveal **patterns hidden in overall data** — e.g., Grade A students may study 6+ hrs while Grade D students study <2 hrs, which overall mean would hide.

QUICK REVISION CHEATSHEET

`df.groupby("col")["num"].agg(...)` → grouped statistics
`.agg(Mean="mean", Std="std", ...)` → multiple stats at once
`df.describe()` → 8 stats for all numeric cols
`df.quantile([0.25, 0.50, 0.75])` → specific percentiles
`df["col"].mean()` → mean of column
`df["col"].std()` → standard deviation
`df["col"].median()` → median
`df[df["species"]=="setosa"]` → filter by category
`.drop(columns="species")` → remove text col before stats
`pd.Categorical.from_codes()` → int codes → category labels

CENTRAL TENDENCY: Mean, Median, Mode, Mid-range

DISPERSION: Range, Variance, SD, Mean Deviation

How to Present to the Examiner

PART 1:

"I created a student dataset with 60 records. The categorical variable is Grade_Group — A, B, or C. I used `groupby()` along with `agg()` to compute mean, median, minimum, maximum, and standard deviation of Score and StudyHrs for each grade group. I also created a dictionary that stores a list of scores for each grade group as required by the question."

PART 2:

"For the Iris dataset, I loaded it from a public GitHub URL. It contains 150 records across 3 species — Iris-setosa, Iris-versicolor, and Iris-virginica. I filtered each species using a boolean condition and applied `describe()` which gives all basic statistics in one shot. I also computed custom percentiles at 10th, 25th, 50th, 75th, and 90th levels using `quantile()`. This helps us understand the distribution of each measurement within each species."

 If the examiner asks "what does percentile tell you?"

"The 25th percentile means 25% of values fall below that number. It helps us understand the spread and distribution of data without being affected by outliers, unlike mean."

 If the examiner asks "why use groupby?"

"groupby() lets us analyse how a numeric variable behaves differently across categories. For example, do Grade-A students study more hours than Grade-C students? groupby gives that answer in one line."

A4: Data Analytics I

 **Practical Title:** Group B — Data Analytics I | Linear Regression | Boston Housing Dataset

 **Problem Statement:** Build a Linear Regression model using Python to predict house prices (MEDV) using the Boston Housing Dataset containing 506 samples and 14 feature variables. Evaluate the model using MAE, MSE, RMSE, and R^2 score.

Linear Regression — Boston Housing

PART 1: LINE-BY-LINE CODE EXPLANATION

BLOCK 1 — Import Libraries

```
import pandas as pd
import numpy as np
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.preprocessing import StandardScaler
```

Import	Purpose
<code>as</code>	Frame operations, data handling
<code>y</code>	Operations, sqrt for RMSE
<code>h_openml</code>	Fetch Boston dataset online (since removed from sklearn)
<code>train_test_split</code>	Split data into training and testing sets
<code>LinearRegression</code>	Linear regression model itself
<code>mean_squared_error, mean_absolute_error, r2_score</code>	Evaluation metrics
<code>StandardScaler</code>	Standardize features to mean=0, std=1

BLOCK 2 — Load Dataset

```
boston = fetch_openml(name="boston", version=1, as_frame=True, parser="auto")
df = boston.frame
df.rename(columns={"MEDV": "HousePrice"}, inplace=True)
```

Line	What it does
<code>fetch_openml(...)</code>	Fetches Boston dataset from OpenML repository

```
frame=True
```

reads data as pandas DataFrame directly

```
as_frame
```

reads the complete DataFrame including target column

```
rename(columns={"MEDV": "HousePrice"})
```

renames target column to readable name

⚠ **EXAM NOTE:** Boston dataset was **removed from sklearn in v1.2** due to ethical concerns about a racial variable (column B). So we use `fetch_openml` instead of `load_boston()`.

If CSV is available on exam PC:

```
df = pd.read_csv("boston.csv")
df.rename(columns={"MEDV": "HousePrice"}, inplace=True)
```

◆ BLOCK 3 — Feature Description

```
features_info = {
    "CRIM" : "Per capita crime rate by town",
    "RM"   : "Average number of rooms per dwelling",
    "LSTAT" : "% lower status of the population",
    "HousePrice": "TARGET"
    # ...etc
}
```

Key Feature	Description	Effect on Price
RM	rooms per dwelling	more rooms → Higher price
LSTAT	lower status population	lower % → Lower price
CRIM	crime rate	higher crime → Lower price
NOX	nitric oxide (pollution)	more pollution → Lower price
DIS	distance to employment	farther → Lower price
HousePrice	GET variable	what we predict

◆ BLOCK 4 — Data Preprocessing

```
print(df.isnull().sum().sum(), "missing values found")
print(df.describe().round(2))
```

- `isnull().sum().sum()` → first `.sum()` per column, second `.sum()` totals everything → **single number** of all missing values
- `describe()` → quick statistical overview of all 14 columns

```
X = df.drop(columns=["HousePrice"])
y = df["HousePrice"].astype(float)
```

- **X = Feature matrix** — all columns except target (13 features, 506 rows)
- **y = Target vector** — only HousePrice column
- `.astype(float)` → ensures target is float for regression calculations

◆ BLOCK 5 — Feature Scaling

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

What StandardScaler does:

$$X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

- Subtracts mean (μ), divides by standard deviation (σ)
- Every feature becomes: **mean = 0, std = 1**

Why scaling is necessary:

- CRIM ranges 0–89, TAX ranges 187–711, RM ranges 3–9
- Without scaling, large-range features **dominate** the model
- Scaling puts all features on **equal footing**

StandardScaler vs MinMaxScaler:

- StandardScaler → mean=0, std=1 → better when outliers present
- MinMaxScaler → range [0,1] → better when no outliers, bounded range needed

◆ BLOCK 6 — Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
```

Parameter	Value	Meaning
<code>test_size=0.2</code>		data = test set (~101 rows)
<code>random_state=42</code>		fixed seed → reproducible split
training set	80% of rows	80% of rows → model learns from this
validation set	20% of rows	20% of rows → model evaluated on this

Why split at all? If we test on the same data we trained on, the model will **cheat** — it already memorized the answers. Test set simulates **unseen real-world data**.

◆ BLOCK 7 — Build & Train Model

```
model = LinearRegression()
model.fit(X_train, y_train)
```


- `LinearRegression()` → creates model object with default parameters
- `.fit(X_train, y_train)` → **trains** the model
 - Internally finds the best coefficients (β values) using **Ordinary Least Squares (OLS)**
 - OLS minimizes the sum of squared differences between actual and predicted values

The Linear Regression Equation:

$$\text{HousePrice} = \beta_0 + \beta_1 \times \text{CRIM} + \beta_2 \times \text{ZN} + \beta_3 \times \text{INDUS} + \dots + \beta_{13} \times \text{LSTAT}$$

- β_0 = **intercept** (value when all features = 0)
- $\beta_1 \dots \beta_{13}$ = **coefficients** (how much each feature affects price)

```
coeff_df = pd.DataFrame({
    "Feature" : df.drop(columns=["HousePrice"]).columns,
    "Coefficient": model.coef_.round(3)
}).sort_values("Coefficient", ascending=False)
```

- `model.coef_` → array of 13 coefficients, one per feature
- `model.intercept_` → the β_0 value
- Sorted by coefficient → shows **most positively and negatively influential features**
- Positive coefficient → feature increases price
- Negative coefficient → feature decreases price

◆ BLOCK 8 — Predictions

```
y_pred = model.predict(X_test)
```

```
comparison = pd.DataFrame({
    "Actual" : y_test.values[:10].round(2),
    "Predicted" : y_pred[:10].round(2),
    "Difference": (y_test.values[:10] - y_pred[:10]).round(2)
})
```

- `model.predict(X_test)` → applies learned equation to test features → produces predicted prices
- Comparison table shows **Actual vs Predicted vs Error** for first 10 rows
- Difference = **residual** (how far off each prediction is)

◆ BLOCK 9 — Model Evaluation Metrics

```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

All 4 metrics explained:

Metric	Formula	What it means	Unit
--------	---------	---------------	------

MAE	$\sum actual - predicted $	average absolute error	same units as target (\$1000s)
MSE	$\frac{1}{n} \sum ((actual - predicted)^2)$	minimizes large errors heavily	same units as target (\$1000s)
R ²	$1 - \frac{SS_{res}}{SS_{tot}}$	coefficient of MSE, interpretable	same units as target (\$1000s)
		variance explained by model	0 to 1 (higher=better)

Interpreting R²:

- $R^2 = 0.75 \rightarrow$ model explains **75% of variance** in house prices
- $R^2 = 1.0 \rightarrow$ perfect prediction
- $R^2 = 0.0 \rightarrow$ model no better than predicting the mean
- $R^2 < 0 \rightarrow$ model is **worse** than just predicting mean



PART 2: ALL POSSIBLE VIVA QUESTIONS & ANSWERS

Q1. What is Linear Regression?

Linear Regression is a **supervised machine learning algorithm** that models the relationship between a dependent variable (target) and one or more independent variables (features) by fitting a **straight line** (or hyperplane in multiple dimensions) through the data. Equation: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$

Q2. What is the difference between Simple and Multiple Linear Regression?

- **Simple Linear Regression:** One input feature $\rightarrow$ one output. $y = \beta_0 + \beta_1 x$
- **Multiple Linear Regression:** Multiple input features $\rightarrow$ one output. $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots$ Boston Housing uses **Multiple** Linear Regression (13 features).

Q3. What is a target variable?

The target variable (also called dependent variable or output variable) is the column we are **trying to predict**. Here it is **HousePrice** (MEDV — Median value of homes).

Q4. What is train-test split? Why do we need it?

We divide the dataset into:

- **Training set** — model learns patterns from this (80%)
- **Testing set** — model evaluated on unseen data (20%) Without this split, the model would be evaluated on data it already saw, giving **falsely optimistic results** (overfitting).

Q5. What does `model.fit()` do?

It **trains** the model — finds the optimal coefficient values ($\beta_0, \beta_1, \dots, \beta_n$) that minimize the error between predicted and actual values using Ordinary Least Squares.

Q6. What does `model.predict()` do?

Takes new input features (X_{test}) and applies the learned equation to **generate predictions** for the target variable.

Q7. What is Ordinary Least Squares (OLS)?

OLS is the method Linear Regression uses to find the best-fit line. It minimizes the **Sum of Squared Residuals (SSR)**: $SSR = \sum(\text{actual} - \text{predicted})^2$
Squaring ensures positive values and penalizes larger errors more.

Q8. What is R^2 score? What is a good R^2 value?

R^2 (R-squared) measures the **proportion of variance in the target variable explained by the model**.

- Range: 0 to 1 (can be negative for very bad models)
- $R^2 = 0.75 \rightarrow$ model explains 75% of variance
- Generally: $R^2 > 0.7$ is considered good, > 0.9 is excellent
- **Context matters** — social science data with $R^2=0.5$ may be fine

Q9. What is the difference between MAE, MSE and RMSE?

- **MAE** — Average of absolute errors. Simple, robust to outliers. Every error treated equally.
- **MSE** — Average of squared errors. Penalizes large errors heavily (squaring). Not in original units.
- **RMSE** — Square root of MSE. Same unit as target. Most commonly reported metric. Penalizes large errors more than MAE. Use **RMSE** when large errors are especially undesirable. Use **MAE** for straightforward average error interpretation.

Q10. Why do we use StandardScaler before Linear Regression?

Features have very different ranges (CRIM: 0-89, TAX: 187-711). Without scaling:

- Features with large values dominate coefficient calculations
- Gradient-based optimizers converge slowly StandardScaler makes all features have **mean=0 and std=1**, ensuring fair contribution from all features.

Q11. What does a positive vs negative coefficient mean?

- **Positive coefficient** (e.g., $RM = +3.5$): As RM increases by 1 unit, HousePrice increases by 3.5 units (more rooms $\rightarrow$ higher price)
- **Negative coefficient** (e.g., $LSTAT = -3.8$): As LSTAT increases by 1, HousePrice decreases by 3.8 units (more lower-status population $\rightarrow$ lower price)

Q12. What is `random_state=42` in `train_test_split`?

Sets a **fixed random seed** so the same data points always go into train and test sets every time the code runs. Without it, different runs give different splits $\rightarrow$ different results $\rightarrow$ not reproducible.

Q13. What are the assumptions of Linear Regression?

1. **Linearity** — relationship between X and y is linear
2. **Independence** — observations are independent of each other
3. **Homoscedasticity** — constant variance of residuals
4. **Normality** — residuals are normally distributed
5. **No Multicollinearity** — features are not highly correlated with each other

Q14. What is Overfitting and Underfitting?

	Overfitting	Underfitting
Training error	low	high
Test error	high	high
Model	Model is too complex, memorizes noise	Model is too simple, misses patterns
Solution	Regularization, more data, simpler model	Add features, complex model
Analogy	Memorized textbook, fails new questions	Don't study at all

Q15. What is Regularization? What are Ridge and Lasso?

Regularization adds a **penalty term** to prevent overfitting by discouraging large coefficients:

- **Ridge (L2)**: Adds penalty = $\lambda \times \sum(\beta^2)$ → shrinks coefficients but never to zero → `Ridge(alpha=1.0)`
- **Lasso (L1)**: Adds penalty = $\lambda \times \sum|\beta|$ → can shrink coefficients **to exactly zero** → performs **feature selection** → `Lasso(alpha=1.0)`
- **ElasticNet**: Combination of both L1 and L2

Q16. What is the difference between Regression and Classification?

	Regression	Classification
Output	Continuous numeric value	Discrete category/class
Example	Predict house price (\$)	Predict if tumor is malignant/benign
Algorithms	Linear Regression, SVR	Logistic Regression, Decision Tree, Naive Bayes
Metrics	MAE, MSE, RMSE, R ²	Accuracy, Precision, Recall, F1

Q17. What is Multicollinearity? How does it affect Linear Regression?

Multicollinearity occurs when **two or more features are highly correlated** with each other (e.g., TAX and RAD in Boston dataset). Effects:

- Coefficients become **unstable and unreliable**
- Small data changes cause large coefficient swings
- Hard to determine individual feature effects
Detection: **VIF (Variance Inflation Factor)** — VIF > 10 indicates serious multicollinearity
Fix:

Remove one of the correlated features, use PCA, or use Ridge regression

Q18. Why was Boston dataset removed from sklearn?

The dataset contains column **'B'** = $1000(B_k - 0.63)^2$ where B_k = proportion of Black residents — a racially discriminatory variable. Sklearn removed it in version 1.2 due to **ethical concerns** about using race as a feature to predict housing prices. The standard replacement is the **California Housing dataset** (`fetch_california_housing()`).

Q19. What is the difference between `fit_transform()` and `transform()`?

- `fit_transform()` on **training data** → learns mean & std from training data AND scales it
- `transform()` on **test data** → uses the **same mean & std** learned from training data to scale test data. If you use `fit_transform()` on test data, it learns different statistics → **data leakage** → incorrect evaluation.

Q20. What are Residuals? What should a good residual plot look like?

A residual is the difference between **actual and predicted value**: `residual = actual - predicted`. In a good model:

- Residuals should be **randomly scattered** around zero
- No pattern (no curve, no funnel shape)
- Approximately normally distributed. Patterns in residuals indicate the model is **missing something** — non-linearity, heteroscedasticity, etc.

Q21. What is the difference between Supervised and Unsupervised Learning?

	Supervised	Unsupervised
Labels	target/output labels	no labels
Goal	learn input→output mapping	discover hidden patterns/structure
Examples	Linear Regression, Classification	K-Means, Hierarchical Clustering
California Housing	Supervised (predicting price)	

⚙️ Preprocessing Steps

Step	Why
<code>StandardScaler()</code>	All features have different scales (crime rate vs tax vs rooms). Scaling brings them to same range so model isn't biased

`train_test_split(test_size=0.2)` 80% data to train the model, 20% to test how well it predicts unseen data

How Linear Regression Works (say this to examiner)

Linear Regression finds the best-fit line/equation:

$\text{HousePrice} = b_0 + b_1 \times \text{CRIM} + b_2 \times \text{RM} + b_3 \times \text{LSTAT} + \dots + b_{13} \times \text{PTRATIO}$

- b_0 = intercept (base price)
 - $b_1, b_2, \dots$ = coefficients (how much each feature affects price)
 - **Positive coefficient** → feature increases price (e.g., more rooms = higher price)
 - **Negative coefficient** → feature decreases price (e.g., more crime = lower price)
-

Evaluation Metrics Explained

Metric	Formula	Meaning
MAE	avg of actual - predicted	Average error in \$1000s — easy to understand
MSE	avg of (actual - predicted) ²	Penalizes large errors more heavily
RMSE	$\sqrt{\text{MSE}}$	Same unit as target (most interpretable)
R² Score	$1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$	% of price variation explained by model

R² closer to 1.0 = better model. A score of ~0.70 means the model explains 70% of house price variation.

How to Present to the Examiner

Step 1 — Dataset:

"I used the Boston Housing dataset loaded directly from scikit-learn's OpenML interface. It has 506 samples and 14 features. The target variable is MEDV — the median house price in thousands of dollars."

Step 2 — Preprocessing:

"I applied StandardScaler to normalize all features to the same scale, which is important for linear regression to work correctly. Then I split the data 80-20 into training and testing sets."

Step 3 — Model:

"I used scikit-learn's LinearRegression class. After fitting it on the training data, I checked the coefficients — features like RM (rooms) have positive coefficients meaning more rooms increases price, while LSTAT (lower status population %) has a negative coefficient meaning it decreases price."

Step 4 — Evaluation:

"I evaluated the model using MAE, MSE, RMSE, and R^2 score. The R^2 score tells us what percentage of house price variation is explained by the model. A value around 0.70 or higher is a good result for this dataset."

If examiner asks "why Linear Regression?"

"Linear Regression is the simplest and most interpretable model for predicting a continuous numeric target like house price. It assumes a linear relationship between features and the target, and the coefficients directly tell us how each feature impacts the price."

If examiner asks "what is RMSE?"

"RMSE is Root Mean Squared Error — it measures average prediction error in the same unit as the target variable. So if RMSE is 4.5, our predictions are on average off by \$4,500 from the actual price."

A5: Data Analytics 2

Logistic Regression Practical (Social Network Ads Dataset)



LINE-BY-LINE CODE EXPLANATION



BLOCK 1 — Import Libraries

```
import pandas as pd
import numpy as np
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    classification_report
)
```

Line	What it does	Why we need it
<code>import pandas as pd</code>	Loads Pandas library	To create, read, clean, and manipulate tabular data
<code>import numpy as np</code>	Loads NumPy library	For numerical operations and random dataset generation
<code>train_test_split</code>	Splits dataset into training and testing sets	To evaluate model on unseen data
<code>LogisticRegression</code>	Imports Logistic Regression algorithm	Used for binary classification
<code>StandardScaler</code>	Imports feature scaling tool	Standardizes features to same scale
<code>confusion_matrix</code>	Computes confusion matrix	To evaluate classification results
<code>accuracy_score</code>	Computes accuracy	Measures overall correctness
<code>precision_score</code>	Computes precision	Measures positive prediction correctness

<code>recall_score</code>	Computes recall	Measures detection of actual positives
<code>classification_report</code>	Generates full performance report	Gives precision, recall, F1-score

To examiner:

“Pandas handles DataFrames, NumPy handles numerical operations and dataset generation, and Scikit-learn provides machine learning algorithms, preprocessing, and evaluation metrics.”

BLOCK 2 — Create Dataset

```
def create_social_network_dataset(filename="Social_Network_Ads.csv",
                                n=100, seed=42):
```

What it does

Creates a custom Social Network Ads dataset and saves it as CSV

Dataset Generation

```
np.random.seed(seed)
```

What it does

Fixes randomness

Why

Same dataset generated every run

```
age = np.random.randint(18, 60, n)
```

What it does

Generates random ages between 18 and 60

```
salary = np.random.randint(15000, 150000, n)
```

What it does

Generates random salaries

```
gender = np.random.choice(["Male", "Female"], n)
```

What it does

Randomly assigns gender

BLOCK 3 — Purchase Logic

```
age_norm = (age - 18) / (60 - 18)
salary_norm = (salary - 15000) / (150000 - 15000)
```

Normalization Formula

$$X_{\text{new}} = \frac{X - X_{\text{min}}}{X_{\text{max}} - X_{\text{min}}}$$

Why normalization?

Converts values into range
0–1

```
score = 0.5 * age_norm + 0.5 * salary_norm
```

What it does

Combines age and salary into purchase
score

```
noise = np.random.normal(0, 0.15, n)
```

What it does

Adds realistic
randomness

```
purchased = (score + noise > 0.55).astype(int)
```

What it does

Converts probability into binary class

Output

1 → Purchased

0 → Not
Purchased

BLOCK 4 — Create DataFrame

```
df = pd.DataFrame({
    "UserID": range(1, n + 1),
    "Gender": gender,
    "Age": age.astype(float),
```

```
"EstimatedSalary": salary.astype(float),  
"Purchased": purchased  
})
```

Column	Meaning
UserID	Unique user identifier
Gender	Male/Female
Age	Age of user
EstimatedSalary	Annual salary
Purchased	Target variable

BLOCK 5 — Missing Values

```
df.loc[missing_age, "Age"] = np.nan
```

What it does

Inserts missing values into Age column

Why Missing Values?

Real-world data often contains:

- incomplete forms
 - sensor failure
 - user skipping information
-

BLOCK 6 — Save CSV

```
df.to_csv(filename, index=False)
```

What it does

Saves dataset locally as CSV file

To examiner:

“The dataset is stored locally so it can be reused offline.”

BLOCK 7 — Load Dataset

```
df = pd.read_csv("Social_Network_Ads.csv")
```

What it does

Loads CSV dataset into
DataFrame

BLOCK 8 — Missing Value Handling

```
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

Mean Formula

$$\text{Mean} = \frac{x_1 + x_2 + x_3 + \dots + x_n}{n}$$

Why mean?

Age and salary are numeric
columns

BLOCK 9 — Features and Target

```
X = df[["Age", "EstimatedSalary"]]  
y = df["Purchased"]
```

Variable	Meaning
X	Features/Input
y	Target/Output

BLOCK 10 — Feature Scaling

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

Standardization Formula

$$z = \frac{x - \mu}{\sigma}$$

$$z = \frac{x - \mu}{\sigma} \approx 1.2 \quad \Phi(z) \approx 88.5\%$$

Where:

- x = original value
 - μ = mean
 - σ = standard deviation
-

Why Scaling?

Without scaling:

- Salary dominates Age
 - Model becomes biased
-

BLOCK 11 — Train-Test Split

```
train_test_split(
    X_scaled,
    y,
    test_size=0.25,
    random_state=0
)
```

Parameter	Meaning
test_size=0.25	25% testing data
random_state=0	Same output every run

BLOCK 12 — Logistic Regression Model

```
model = LogisticRegression()
model.fit(X_train, y_train)
```

What it does

Trains Logistic Regression model

Logistic Regression Formula

Linear Equation

$$z = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$
$$z = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

Sigmoid Function

$$P(y=1) = \frac{1}{1 + e^{-z}}$$

Threshold Rule

Probability > 0.5 → Purchased = 1
Probability < 0.5 → Purchased = 0

BLOCK 13 — Predictions

```
y_pred = model.predict(X_test)
```

What it does

Predicts Purchased or Not
Purchased

BLOCK 14 — Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
```

Confusion Matrix Structure

	Predicted YES	Predicted NO
Actual YES	TP	FN
Actual NO	FP	TN

TP, TN, FP, FN

Term	Meaning
TP	Predicted YES, actually YES
TN	Predicted NO, actually NO

FP Predicted YES, actually NO

FN Predicted NO, actually YES

BLOCK 15 — Evaluation Metrics

Accuracy Formula

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Error Rate Formula

$$\text{Error Rate} = 1 - \text{Accuracy}$$

Precision Formula

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall Formula

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1 Score Formula

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

BLOCK 16 — Classification Report

`classification_report()`

Gives

Precision

Recall

F1-score

Support



IMPORTANT VIVA QUESTIONS

Q1. Why Logistic Regression?

Used for binary classification problems.

Q2. Why not Linear Regression?

Linear Regression predicts continuous values.
Logistic Regression predicts categories/classes.

Q3. Why scaling is important?

Features are on different scales.

Q4. Why confusion matrix?

To evaluate classification performance in detail.

Q5. Why use train-test split?

To test model on unseen data.

Q6. Why use StandardScaler?

To standardize features.

Q7. Why use sigmoid function?

To convert output into probability.

Q8. Why missing values were added?

To simulate real-world datasets.

REAL-WORLD APPLICATIONS

Application	Example
Medical diagnosis	Disease / No disease
Spam detection	Spam / Not spam
Fraud detection	Fraud / Genuine
Advertisement prediction	Buy / Not buy
Loan approval	Approved / Rejected
Student performance	Pass / Fail

ONE-MINUTE PRACTICAL EXPLANATION

“This practical implements Logistic Regression on a self-created Social Network Ads dataset. The dataset contains Age and EstimatedSalary as features and Purchased as the target variable. Missing values are handled using mean imputation. Features are standardized using StandardScaler. The data is split into training and testing sets. Logistic Regression predicts whether a customer will purchase a product or not using the sigmoid function. The model is evaluated using confusion matrix, accuracy, precision, recall, and classification report.”

Logistic Regression — Detailed Explanation (Exam + Viva Ready)

1. What is Logistic Regression?

Logistic Regression is a **Supervised Machine Learning Classification Algorithm** used to predict **categorical outcomes**.

It predicts:

- Yes / No
- True / False
- 0 / 1
- Purchased / Not Purchased

In your practical:

- 0 → Not Purchased
 - 1 → Purchased
-

2. Why is it called “Regression” if it is Classification?

This is one of the MOST ASKED viva questions.

Answer:

It is called “Regression” because internally it uses a regression equation:

$$z = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$
$$b_{nx_n} = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

But instead of predicting continuous values directly,
it passes the result through the **Sigmoid Function** to produce probabilities.

Therefore:

- Mathematical base → Regression
 - Final task → Classification
-

3. Main Goal of Logistic Regression

The goal is to estimate:

Probability that an input belongs to a particular class.

Example:

- Probability user buys product = 0.92
- Probability user buys product = 0.15

Then:

- If probability > 0.5 → class = 1
 - Else → class = 0
-

4. Why Logistic Regression is Used in This Practical?

Because the output column (**Purchased**) has only TWO classes:

Value	Meaning
0	Not Purchased
1	Purchased

This is called:

Binary Classification

5. How Logistic Regression Works Internally

VERY IMPORTANT FOR EXAM.

STEP 1 — Take Input Features

Features:

- Age
- EstimatedSalary

Example:

Age Salary

40 90000

STEP 2 — Apply Linear Equation

The algorithm calculates:

$$z = b_0 + b_1x_1 + b_2x_2 \quad z = b_0 + b_1x_1 + b_2x_2$$

Where:

- z = weighted sum
- b_0 = intercept
- b_1, b_2 = coefficients
- x_1, x_2 = feature values

Example:

Age contributes some weight

Salary contributes some weight

The model combines them.

STEP 3 — Apply Sigmoid Function

Linear regression output can be:

-100

+250

+5

But probabilities must stay between:

0 and 1

So Logistic Regression uses:

Sigmoid Function

$$P(y=1) = \frac{1}{1 + e^{-z}} \quad P(y=1) = \frac{1}{1 + e^{-z}}$$

Why Sigmoid Function?

Because it converts ANY number into:

0 to 1

Example:

z	Sigmoid Output
-10	0.00004
0	0.5
+10	0.999

STEP 4 — Apply Threshold

Usually threshold = 0.5

Rule:

If probability > 0.5 → Purchased = 1

Else → Purchased = 0

Example:

Probability	Prediction
0.87	Purchased
0.12	Not Purchased

6. Sigmoid Curve

VERY IMPORTANT THEORY QUESTION.

The sigmoid function creates an:

S-shaped curve

Properties:

- Output always between 0 and 1
 - Smooth curve
 - Converts regression output into probability
-

7. Why Feature Scaling is Needed?

Your practical uses:

StandardScaler()

Because:

- Age range → 18–60
- Salary range → 15000–150000

Without scaling:

- Salary dominates calculations
 - Model becomes biased
-

Standardization Formula

$$z = \frac{x - \mu}{\sigma}$$

xxx

μ

σ

$$z = \frac{x - \mu}{\sigma} \approx 1.2$$

$$\Phi(z) \approx 88.5\%$$

Where:

- x = original value
 - μ = mean
 - σ = standard deviation
-

8. Cost Function in Logistic Regression

VERY IMPORTANT THEORY.

Logistic Regression cannot use Mean Squared Error like Linear Regression.

Instead it uses:

Log Loss / Cross Entropy Loss

Formula:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y(i) \log(h_{\theta}(x(i))) + (1 - y(i)) \log(1 - h_{\theta}(x(i)))]$$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y(i) \log(h_{\theta}(x(i))) + (1 - y(i)) \log(1 - h_{\theta}(x(i)))]$$

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y(i) \log(h_{\theta}(x(i))) + (1 - y(i)) \log(1 - h_{\theta}(x(i)))]$$

Why not MSE?

Because:

- Sigmoid creates non-convex behavior
- Gradient descent becomes difficult
- Optimization becomes inefficient

Log Loss works better for probabilities.

9. Gradient Descent

Used to optimize coefficients.

Goal:

- Minimize error
- Improve predictions

Formula:

$$\theta := \theta - \alpha \frac{\partial J(\theta)}{\partial \theta} \quad \theta := \theta - \alpha \frac{\partial J(\theta)}{\partial \theta}$$

Where:

- θ = parameters
 - α = learning rate
 - $J(\theta)$ = cost function
-

10. Decision Boundary

Logistic Regression creates a boundary separating classes.

Example:

Left side → Not Purchased

Right side → Purchased

Boundary usually occurs at:

Probability = 0.5

11. Advantages of Logistic Regression

Advantage	Explanation
Simple	Easy to implement
Fast	Training is efficient
Probability output	Gives probability values
Interpretable	Coefficients understandable
Good for binary classification	Excellent for 0/1 tasks

12. Limitations of Logistic Regression

Limitation	Explanation
Linear boundary	Cannot model complex nonlinear patterns
Sensitive to outliers	Extreme values affect model
Needs scaling	Different scales create issues
Assumes linear relationship	Between features and log odds

13. Real-World Applications

Application	Example
Medical diagnosis	Disease / No disease
Spam filtering	Spam / Not spam
Fraud detection	Fraud / Genuine
Advertisement prediction	Buy / Not buy
Loan approval	Approved / Rejected
Student performance	Pass / Fail

14. FULL ALGORITHM FOR EXAM WRITING

Algorithm: Logistic Regression on Social Network Ads Dataset

Step 1:

Import required libraries such as Pandas, NumPy, Scikit-learn.

Step 2:

Create or load the Social Network Ads dataset.

Step 3:

Display dataset information and check missing values.

Step 4:

Handle missing values using mean imputation.

Step 5:

Select feature variables (*Age*, *EstimatedSalary*) and target variable (*Purchased*).

Step 6:

Apply feature scaling using *StandardScaler*.

Step 7:

Split the dataset into training and testing sets using *train_test_split()*.

Step 8:

Create Logistic Regression model using *LogisticRegression()*.

Step 9:

Train the model using *fit()* method.

Step 10:

Predict output using *predict()* method.

Step 11:

Generate confusion matrix using `confusion_matrix()`.

Step 12:

Calculate Accuracy, Precision, Recall, Error Rate, and Classification Report.

Step 13:

Interpret results and evaluate model performance.

15. VERY IMPORTANT EXTERNAL QUESTIONS

Q1. Why Logistic Regression?

Because output variable is categorical/binary.

Q2. Why not Linear Regression?

Linear Regression predicts continuous values.

Logistic Regression predicts probabilities/classes.

Q3. Why Sigmoid Function?

To convert output into probability between 0 and 1.

Q4. Why Scaling?

To prevent large-scale features from dominating.

Q5. Why Confusion Matrix?

To evaluate classification performance.

Q6. What is Threshold?

Decision cutoff value.

Usually:

0.5

Q7. Why Train-Test Split?

To test model on unseen data.

Q8. What happens without scaling?

Salary dominates Age.

16. ONE-MINUTE FULL EXPLANATION FOR VIVA

Memorize this:

“Logistic Regression is a supervised machine learning algorithm used for binary classification problems. In this practical, it predicts whether a customer will purchase a product based on Age and EstimatedSalary. The algorithm first calculates a weighted linear equation and then applies the sigmoid function to convert the result into probability between 0 and 1. If the probability is greater than 0.5, the output is classified as Purchased; otherwise Not Purchased. The model is trained using training data and evaluated using confusion matrix, accuracy, precision, recall, and classification report.”

For practical 5 and 6

Explanation — Practical 5 (Logistic Regression)

Why Logistic Regression?

Unlike Linear Regression which predicts a continuous number, **Logistic Regression predicts a probability (0 to 1)** — perfect for binary classification like "Purchased = Yes/No".

Dataset — Social Network Ads

- 400 records of users with Age, EstimatedSalary, Purchased
- Features used: Age + EstimatedSalary → predict Purchased (0 or 1)

Steps done:

Step	Why
<code>StandardScaler</code>	Age (20-60) and Salary (15k-150k) have very different scales — scaling is needed
<code>train_test_split(0.25)</code>	75% train, 25% test
<code>LogisticRegression()</code>	Finds a decision boundary (S-curve) to classify users

✖ Explanation — Practical 6 (Naïve Bayes)

Why Gaussian Naïve Bayes?

Naïve Bayes is a **probabilistic classifier** — it calculates the probability of each class given the input features using Bayes' theorem. `GaussianNB` is used specifically because Iris features are **continuous float values** (not counts or binary).

Naïve assumption: All features are independent of each other — hence "Naïve".

Multi-class Confusion Matrix:

Iris has 3 classes, so the confusion matrix is **3×3** instead of 2×2. For calculating TP/FP/TN/FN per class, we use **One-vs-Rest** strategy:

- Pick one class as "Positive"
 - All others combined = "Negative"
 - Calculate TP, FP, FN, TN for that class
 - Repeat for all 3 classes
-

📊 Confusion Matrix — Formulas (must know for examiner)

	Predicted YES	Predicted NO
Actual YES	TP	FN
Actual NO	FP	TN

Metric	Formula	Meaning
Accuracy	$(TP+TN) / \text{Total}$	Overall correct predictions
Error Rate	$1 - \text{Accuracy}$	Overall wrong predictions
Precision	$TP / (TP+FP)$	Of all predicted positive, how many were actually positive
Recall	$TP / (TP+FN)$	Of all actual positive, how many did we catch

🎤 How to Present to Examiner

Practical 5:

"I used Logistic Regression to predict whether a social network user purchased a product based on their Age and EstimatedSalary. After scaling the features and splitting the data, I trained the model and computed the confusion matrix. From the matrix, I extracted TP, TN, FP, FN and calculated Accuracy, Error Rate, Precision, and Recall manually."

Practical 6:

"I implemented Gaussian Naïve Bayes on the Iris dataset to classify flowers into 3 species. I used GaussianNB because the feature values are continuous. Since this is a multi-class problem, the confusion matrix is 3×3. I computed TP, FP, FN, TN for each class separately using the One-vs-Rest approach, then computed overall metrics using macro averaging."

If examiner asks "difference between Logistic Regression and Naïve Bayes?"

"Logistic Regression is a discriminative model — it directly learns the decision boundary. Naïve Bayes is a generative model — it learns the probability distribution of each class. Naïve Bayes is faster and works well with small data, while Logistic Regression is more accurate when features are correlated."

If examiner asks "what is macro averaging?"

"Macro averaging computes the metric for each class separately and then takes a simple average. It treats all classes equally regardless of their size — suitable when all classes are equally important like in the Iris dataset."

A6: Data Analytics 3

Naive Bayes Classification — Detailed Explanation (Exam + Viva Ready)

This practical implements **Naive Bayes Classification** on the Iris dataset to classify flowers into:

- Setosa
 - Versicolor
 - Virginica
-

1. What is Naive Bayes?

Naive Bayes is a:

Supervised Machine Learning Classification Algorithm

It is based on:

Bayes Theorem

Used for:

- Classification problems
 - Probability prediction
 - Multi-class classification
-

2. Why “Naive”?

The algorithm assumes:

All features are independent of each other.

Example:

- Sepal length independent of petal width
- Petal length independent of sepal width

This assumption is usually not fully true in real life.

That's why it is called:

“Naive”

3. Why “Bayes”?

Because it uses:

Bayes Theorem

to calculate probabilities.

4. Main Goal of Naive Bayes

To calculate:

Probability that a flower belongs to a particular species.

Example:

Species	Probability
sa	
icolor	
nica	

Prediction:

Highest probability wins

5. What Type of Problem is This?

This is:

Multi-Class Classification

Because:

- 3 classes exist
- setosa
- versicolor
- virginica

Unlike Logistic Regression:

0 / 1 only

Naive Bayes handles multiple classes naturally.

6. What Dataset is Used?

Iris Dataset from sklearn.

Contains:

- 150 rows
 - 4 numeric features
 - 3 flower species
-

7. Features Used

Feature	Meaning
sepal length	length of sepal
sepal width	width of sepal
petal length	length of petal
petal width	width of petal

8. Target Variable

species

Possible values:

- setosa
 - versicolor
 - virginica
-

9. Types of Naive Bayes

Type	Used For
GaussianNB	Continuous numeric data
MultinomialNB	Count data
BernoulliNB	Binary features

10. Which Naive Bayes is Used Here?

GaussianNB()

Because:

- Features are continuous numeric values
- Assumes Gaussian (normal) distribution

11. Bayes Theorem

MOST IMPORTANT FORMULA.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \quad P(A|B) = \frac{P(B|A)P(A)}{P(A)P(A)}$$

$$P(B|A)P(B|A)P(B|A)$$

$$P(B \mid \neg A)P(B \mid \neg A)P(B \mid \neg A)$$

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{0.16}{0.25} = 0.64$$

$$P(B) \approx 0.25, P(A|B) = P(B)P(B|A)P(A) \approx 0.68, P(B) \approx 0.25$$

$P(B)=0.25P(B|A)P(A)=0.17P(A|B)\sim 0.68$ Posterior = useful evidence / total evidence

Meaning of Terms

Symbol	Meaning
B)	erior Probability
A)	elihood
	Probability
	ance

In This Practical

A = flower species

B = feature values

Example:

$P(\text{Setosa} \mid \text{petal_length}=1.5)$

means:

Probability flower is Setosa given petal length is 1.5

12. How Naive Bayes Works

VERY IMPORTANT FOR EXAM.

STEP 1 — Load Dataset

Dataset loaded from sklearn:

```
datasets.load_iris()
```

STEP 2 — Separate Features and Target

```
X = df.drop(columns=["species"])
```

```
y = df["species"]
```

Features:

- Numeric measurements

Target:

- Species
-

STEP 3 — Split Training and Testing Data

```
train_test_split()
```

Purpose:

- Train on one part
- Test on unseen data

STEP 4 — Train GaussianNB Model

```
model = GaussianNB()
model.fit(X_train, y_train)
```

STEP 5 — Calculate Probabilities

The model computes:

$$P(\text{class} | \text{features}) \propto P(\text{features} | \text{class}) \times P(\text{class})$$

$$P(\text{class} | \text{features}) \propto P(\text{features} | \text{class}) \times P(\text{class})$$

Where:

- $P(\text{class}) \rightarrow$ prior probability
 - $P(\text{features} | \text{class}) \rightarrow$ likelihood
-

STEP 6 — Choose Highest Probability

Example:

Species	Probability
sa	
icolor	
nica	

Prediction:

Setosa

13. Prior Probability

Your code prints:

```
model.class_prior_
```

What is Prior Probability?

Probability of class before seeing data.

Formula:

$$P(A) = \frac{\text{Number of samples in class A}}{\text{Total samples}}$$

Example:

50 setosa out of 150

$$P(\text{setosa}) = 50/150 = 0.33$$

14. Gaussian Distribution

GaussianNB assumes:

Features follow normal distribution

Normal Distribution Formula:

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Where:

- μ = mean
- σ = standard deviation

15. Predictions

```
y_pred = model.predict(X_test)
```

The model predicts:

- setosa
- versicolor
- virginica

16. Confusion Matrix

`confusion_matrix(y_test, y_pred)`

Confusion Matrix Meaning

Rows:

Actual class

Columns:

Predicted class

Diagonal values:

Correct predictions

Off-diagonal:

Wrong predictions

17. TP, FP, FN, TN

In multi-class classification:

- calculated using One-vs-Rest method
-

TP (True Positive)

Correctly predicted as that class.

FP (False Positive)

Predicted as class wrongly.

FN (False Negative)

Actually belongs to class but predicted wrong.

TN (True Negative)

Correctly not predicted as that class.

18. Accuracy

Formula:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

19. Error Rate

Formula:

$$\text{Error Rate} = 1 - \text{Accuracy}$$

20. Precision

Formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

21. Recall

Formula:

$$\text{Recall} = \frac{TP}{TP + FN}$$

22. Macro Average

Your code uses:

```
average="macro"
```

Meaning:

- calculate metric separately for each class
- take simple average

Useful for multi-class classification.

23. Classification Report

```
classification_report()
```

Contains:

- Precision
- Recall
- F1-score
- Support

24. Why Naive Bayes Works Well?

Reason	Explanation
Efficient	Naive Bayes is computationally efficient.
Simple	It requires fewer calculations than other models.
Works well with small data	Naive Bayes performs well on smaller datasets.
Handles multi-class support	Naive Bayes naturally handles multiple classes.
Probabilistic	It uses probability distributions to make predictions.

25. Limitations of Naive Bayes

Limitation	Explanation
Independence assumption	Features may actually depend on each other.
Does not work well with correlated features	Violates the independence assumption.
Zero frequency problem	Zero values create issues in probability calculations.

26. Real-World Applications

Application	Example
Spam filtering	Classifying emails as spam or not spam.
Document classification	Assigning documents to categories like news, sports, etc.
Image classification	Identifying objects in images, like cats or dogs.
Sentiment analysis	Determining if a review is positive or negative.

Spam Detection	Spam / Not Spam
Email Filtering	Spam / Social / Primary
Document Classification	Spam categories
Sentiment Analysis	Positive / Negative
Medical Diagnosis	Disease prediction
Image Classification	Image species

27. FULL ALGORITHM FOR EXAM WRITING

Algorithm: Naive Bayes Classification on Iris Dataset

Step 1:

Import required libraries such as Pandas, NumPy, and Scikit-learn.

Step 2:

Load Iris dataset using `datasets.load_iris()`.

Step 3:

Create DataFrame and add species column.

Step 4:

Display dataset summary and class distribution.

Step 5:

Separate feature variables and target variable.

Step 6:

Split dataset into training and testing sets using `train_test_split()`.

Step 7:

Create Gaussian Naive Bayes model using `GaussianNB()`.

Step 8:

Train model using `fit()` method.

Step 9:

Predict species using `predict()`.

Step 10:

Generate confusion matrix.

Step 11:

Calculate TP, FP, FN, TN, Accuracy, Precision, Recall.

Step 12:

Display classification report and interpret results.

28. VERY IMPORTANT EXTERNAL QUESTIONS

Q1. Why GaussianNB?

Because features are continuous numeric values.

Q2. Why called Naive?

Assumes all features are independent.

Q3. What is Bayes Theorem?

Probability theorem used for classification.

Q4. What is Prior Probability?

Probability before observing data.

Q5. Why suitable for Iris dataset?

Numeric continuous features follow Gaussian distribution.

Q6. Difference between Logistic Regression and Naive Bayes?

Logistic Regression	Naive Bayes
Discriminative model	Generative model
Linear decision boundary	Assumes probability distribution
Requires more data	Works well on small data

29. ONE-MINUTE VIVA EXPLANATION

“This practical implements Gaussian Naive Bayes classification on the Iris dataset. The dataset contains four numeric flower measurements and three species classes. GaussianNB is used because the features are continuous numeric values and assumed to follow Gaussian distribution. The model calculates posterior probabilities using Bayes Theorem and predicts the species with highest probability. The performance is evaluated using confusion matrix, accuracy, precision, recall, and classification report.”

Naive Bayes — Every Important Term Explained in Detail (Viva + Theory Notes)

1. Naive Bayes

Definition

Naive Bayes is a:

Supervised Machine Learning Classification Algorithm

based on:

Probability and Bayes Theorem

Used for:

- Classification
- Probability prediction
- Multi-class problems

Examples:

- Spam detection
 - Disease prediction
 - Sentiment analysis
 - Flower classification
-

Why is it called “Naive”?

Because it assumes:

All features are independent from each other.

Example in Iris dataset:

- Petal length independent of petal width
- Sepal width independent of sepal length

Realistically:

This is usually NOT fully true

But the algorithm still works surprisingly well.

That unrealistic assumption makes it:

“Naive”

Why is it called “Bayes”?

Because it uses:

Bayes Theorem

to calculate probabilities.

Main Idea of Naive Bayes

It predicts:

Which class has the highest probability.

Example:

Species	Probability
sa	
icolor	
nica	

Prediction:

Setosa

2. Bayes Theorem

MOST IMPORTANT THEORY.

Definition

Bayes Theorem calculates:

Probability of an event using prior knowledge.

Formula

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$
$$P(A|B) = P(B)P(B|A)P(A)$$

$P(B|A)P(B|A)P(B|A)$
 $P(B|\neg A)P(B|\neg A)P(B|\neg A)$
 $P(A|B)=\frac{P(B|A)P(A)}{P(B)}\approx 0.68, P(B)\approx 0.25$
 $P(A|B)=\frac{P(B|A)P(A)}{P(B)}\approx 0.68, P(B)\approx 0.25$
 $P(B)=0.25$
 $P(A|B)=\frac{P(B|A)P(A)}{P(B)}\approx 0.68$
 Posterior = useful evidence / total evidence

Meaning of Terms

Symbol	Meaning
$P(A)$	Prior Probability
$P(B A)$	Likelihood
$P(B)$	Marginal Probability
$P(A B)$	Posterior Probability

Real Meaning in Iris Dataset

Suppose:

A = flower is Setosa
 B = petal length = 1.5

Then:

$P(\text{Setosa} | \text{petal length}=1.5)$

means:

Probability flower is Setosa given petal length is 1.5

Bayes Theorem Flow

Naive Bayes calculates:

Posterior Probability
 =
 Likelihood $\times$ Prior Probability

 Evidence

3. Posterior Probability

Definition

Probability AFTER observing data.

Formula:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Example

Probability flower is Setosa
AFTER seeing petal length

Importance

This is the FINAL probability used for prediction.

Highest posterior probability wins.

4. Prior Probability

VERY IMPORTANT.

Definition

Probability BEFORE seeing any data/features.

Formula

$$P(A) = \frac{\text{Number of samples in class A}}{\text{Total samples}}$$
$$P(A) = \frac{\text{Number of samples in class A}}{\text{Total samples}}$$

Example

Suppose:

- 50 Setosa flowers
- 150 total flowers

Then:

$$P(\text{Setosa}) = \frac{50}{150} = 0.33$$

In Your Code

```
model.class_prior_  
  
    prints prior probabilities.
```

Why Prior Probability Important?

It tells:

How common each class is
before analyzing features.

5. Likelihood

Definition

Probability of observing features given the class.

Formula:

$$P(B|A)$$

Example

Probability of petal length = 1.5
IF flower is Setosa

Importance

Measures:

How likely features belong to that class

6. Evidence

Definition

Overall probability of observed data.

Formula:

$P(B)P(B)P(B)$

Purpose

Used for normalization.

Usually ignored during comparison because:

same for all classes

7. Gaussian Naive Bayes (GaussianNB)

VERY IMPORTANT.

Definition

A type of Naive Bayes used for:

Continuous Numeric Features

Examples:

- Height
- Weight
- Age

- Petal length
-

Why “Gaussian”?

Because it assumes:

Features follow Gaussian (Normal) Distribution

Normal Distribution

Also called:

Bell-shaped curve

Properties:

- Symmetrical
 - Mean at center
 - Most values near mean
-

Gaussian Distribution Formula

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

$f(x)$ is the probability density function
 μ is the mean
 σ is the standard deviation

Meaning of Symbols

Symbol	Meaning
μ	Mean value
σ	Standard deviation
e	Euler's constant

In Iris Dataset

Features are:

- sepal length
- sepal width
- petal length
- petal width

These are:

continuous numeric values

Therefore:

GaussianNB()

is used.

8. Independence Assumption

MOST IMPORTANT THEORY QUESTION.

Naive Bayes assumes:

Every feature is independent

Meaning:

- petal length independent of petal width
-

Why This Assumption Helps?

Because it simplifies calculations.

Instead of:

complex joint probability

it multiplies simple probabilities.

Formula with Independence

$$P(x_1, x_2, \dots, x_n | C) = P(x_1 | C)P(x_2 | C) \dots P(x_n | C) \\ P(x_1, x_2, \dots, x_n | C) = P(x_1 | C)P(x_2 | C) \dots P(x_n | C)$$

9. Training

Definition

Process where model learns patterns from data.

In Your Code

```
model.fit(X_train, y_train)
```

What Happens Internally?

Model calculates:

- Mean of each feature per class
 - Standard deviation per class
 - Prior probabilities
-

10. Prediction

Definition

Using learned probabilities to classify new data.

In Your Code

```
model.predict(X_test)
```

Internally Model Does:

For every class:

1. Calculate posterior probability
 2. Compare probabilities
 3. Choose highest probability
-

11. Multi-Class Classification

Definition

Classification with:

more than 2 classes

Example

Iris dataset:

- Setosa
 - Versicolor
 - Virginica
-

Difference from Binary Classification

Binary	Multi-Class
2 classes	more than 2
No	Setosa/Versicolor/Virginica

12. Confusion Matrix

Definition

Table showing:

- Correct predictions

- Wrong predictions
-

Structure

Rows:

Actual classes

Columns:

Predicted classes

Diagonal Values

Correct predictions.

Off-Diagonal Values

Wrong predictions.

13. TP, FP, FN, TN

In multi-class classification:

- calculated One-vs-Rest
-

True Positive (TP)

Correctly predicted class.

False Positive (FP)

Wrongly predicted as class.

False Negative (FN)

Actually class but predicted wrong.

True Negative (TN)

Correctly not predicted as class.

14. Accuracy

Definition

Overall correctness.

Formula:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

15. Precision

Definition

Out of predicted positives,
how many are correct.

Formula:

$$\text{Precision} = \frac{TP}{TP + FP}$$

16. Recall

Definition

Out of actual positives,
how many correctly detected.

Formula:

$$\text{Recall} = \frac{TP}{TP + FN}$$

17. F1-Score

Definition

Balance between precision and recall.

Formula:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

18. Macro Average

Your code uses:

```
average="macro"
```

Meaning:

- Calculate metric for each class separately
- Take simple average

19. Classification Report

Definition

Detailed summary containing:

- Precision
 - Recall
 - F1-score
 - Support
-

Support

Number of actual samples in each class.

20. Why Naive Bayes is Fast?

Because:

- Simple probability calculations
 - Independence assumption reduces complexity
-

21. Advantages of Naive Bayes

Advantage	Explanation
Simple calculations	Simple probability calculations
Easy implementation	Easy implementation
Multi-class support	Handles many classes
Works with small data	Works for small datasets
Robust to missing probabilities	Handles missing probabilities

22. Limitations of Naive Bayes

Limitation	Explanation
Independence assumption unrealistic	Features may correlate
Not accurate with correlated data	Accuracy may decrease
Zero-frequency problem	When feature values create issues

23. Real-World Applications

Application	Example
Spam filtering	Spam / Not spam
Sentiment analysis	Positive / Negative

cal diagnosis	ase prediction
s classification	ts / Politics
iment categorization	; prediction
er classification	pecies

24. MOST IMPORTANT VIVA QUESTIONS

Q1. Why called Naive?

Because assumes all features are independent.

Q2. Why GaussianNB?

Because features are continuous numeric values.

Q3. What is Bayes Theorem?

Probability theorem used for classification.

Q4. What is Prior Probability?

Probability before observing data.

Q5. Why Naive Bayes is fast?

Uses simple probability calculations.

Q6. Why suitable for Iris dataset?

Features are continuous and normally distributed.

Q7. Difference between Logistic Regression and Naive Bayes?

Logistic Regression

Naive Bayes

Discriminative

Generative

Linear boundary

Assumes distributions

Less

Less

Needs more data

Works well on small data

A7:Text Analysis

Text Analysis & NLP Preprocessing Practical

PART 1 — INTRODUCTION TO NLP

What is NLP?

NLP = Natural Language Processing

It is a branch of:

Artificial Intelligence (AI)

that enables computers to:

- understand
- process
- analyze
- generate

human language.

Examples of NLP

Application	Example
Chatbots	ChatGPT
Translation	Google Translate
Voice assistants	Siri, Alexa
Spam filtering	Gmail
Sentiment analysis	Positive/Negative review detection
Text summarization	News summarizers

Main Goal of This Practical

This practical performs:

- Text preprocessing
- Tokenization
- POS tagging
- Stopword removal
- Stemming
- Lemmatization
- TF-IDF feature extraction

using:

- NLTK
 - Scikit-learn
-



BLOCK 1 — Import Libraries

```
import nltk
import pandas as pd
import math
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk import pos_tag
from sklearn.feature_extraction.text import TfidfVectorizer
```

nltk

Definition

NLTK =

Natural Language Toolkit

A Python library used for:

- text processing
 - NLP tasks
 - language analysis
-

pandas

Used for:

- tables
 - DataFrames
 - displaying outputs neatly
-

math

Used for:

- logarithmic calculations
 - TF-IDF manual calculation
-

word_tokenize

Used for:

splitting text into words

Example:

"I love NLP"

becomes:

["I", "love", "NLP"]

sent_tokenize

Used for:

splitting paragraph into sentences

stopwords

Predefined common words like:

- is
- am
- the
- and

These words usually carry little meaning.

PorterStemmer

Used for:

stemming

reducing words to root form.

Example:

Word	Stem
playing	play
studies	studi

WordNetLemmatizer

Used for:

lemmatization

converts words into meaningful dictionary root words.

pos_tag

Used for:

Part-of-Speech Tagging

Identifies:

- noun
- verb
- adjective
- adverb

etc.

TfidfVectorizer

Used for:

TF-IDF Feature Extraction

Converts text into numerical vectors.



BLOCK 2 — Download NLTK Resources

```
nltk.download("punkt")
```

Why downloads needed?

NLTK modules need:

- tokenizer models
 - stopword dictionaries
 - POS tagging models
-

punkt

Used for:

- sentence tokenization
 - word tokenization
-

stopwords

Contains common stopword list.

wordnet

Used by lemmatizer.

averaged_perceptron_tagger

Used for POS tagging.

BLOCK 3 — Sample Document

```
document = ""  
Natural Language Processing is a fascinating field...  
""
```

Document

A block of text used for NLP processing.

PART 1 — TEXT PREPROCESSING

What is Text Preprocessing?

Process of:

cleaning and preparing text

before applying Machine Learning.

Why preprocessing needed?

Raw text contains:

- punctuation
- unnecessary words
- inconsistent formats
- noise

Preprocessing improves:

- model accuracy
 - efficiency
 - feature quality
-

STEP 1 — Sentence Tokenization

```
sent_tokens = sent_tokenize(document)
```

Tokenization

Definition

Process of breaking text into smaller units called:

tokens

Sentence Tokenization

Splits paragraph into:

sentences

Example:

"I love AI. NLP is interesting."

becomes:

```
[  
"I love AI.",  
"NLP is interesting."  
]
```

Why needed?

Useful for:

- summarization
 - chatbot understanding
 - sentence analysis
-

STEP 2 — Word Tokenization

```
word_tokens = word_tokenize(document)
```

Word Tokenization

Splits text into:

words/tokens

Example:

"I love NLP"

becomes:

```
["I", "love", "NLP"]
```

Why important?

Machine Learning models process:

tokens, not paragraphs

STEP 3 — POS Tagging

```
pos_tags = pos_tag(word_tokens)
```

POS Tagging

POS =

Part Of Speech

Identifies grammatical role of words.

Common POS Tags

Tag	Meaning
NN	Noun
NNP	Proper noun
VB	Verb
VBG	Verb gerund
JJ	Adjective
RB	Adverb
DT	Determiner
IN	Preposition

Example

Dogs bark loudly

becomes:

Word	POS
Dogs	NN
bark	VB

Why POS tagging important?

Used in:

- grammar analysis
 - chatbots
 - translation
 - information extraction
-



STEP 4 — Stopword Removal

```
stop_words = set(stopwords.words("english"))
```

Stopwords

Common frequent words:

- the
 - is
 - are
 - and
 - of
-

Why remove stopwords?

Because:

- they carry little meaning
 - increase noise
 - increase dimensionality
-

Example

Before:

"The cat is on the mat"

After:

```
["cat", "mat"]
```

isalpha()

w.isalpha()

Checks:

word contains only alphabets

Removes:

- punctuation
 - numbers
-

STEP 5 — Stemming

```
stemmer = PorterStemmer()
```

Stemming

Process of reducing word into:

root/stem form

Examples

Original	Stemmed
playing	play
studies	studi
running	run

PorterStemmer

A popular stemming algorithm.

Problem with Stemming

Sometimes produces:

non-meaningful words

Example:

studies → studi

Why stemming used?

Reduces vocabulary size.

Words with same meaning treated similarly.



STEP 6 — Lemmatization

```
lemmatizer = WordNetLemmatizer()
```

Lemmatization

Converts word into:

meaningful dictionary root word

Examples

Original	Lemmatized
studies	study
running	run
better	good

Difference Between Stemming and Lemmatization

Stemming	Lemmatization
Faster	Slower
May create invalid words	Produces valid words
Rule-based	Dictionary-based

Why lemmatization better?

Maintains:

- proper meaning
 - valid grammar
-

PART 2 — TF-IDF

MOST IMPORTANT THEORY.

What is TF-IDF?

TF-IDF =

Term Frequency – Inverse Document Frequency

Used to measure:

importance of words in documents

Why TF-IDF needed?

Some words appear frequently:

the, is, and

but are not important.

TF-IDF gives:

- higher weight to important rare words
 - lower weight to common words
-

Corpus

Definition

Collection of documents.

Your code:

```
corpus = [  
    "doc1",  
    "doc2",  
    ...  
]
```

Document

Single text inside corpus.

Term Frequency (TF)

Measures:

how frequently word appears in a document

TF Formula

TF(t)=Number of times term appears in documentTotal words in document
 $TF(t)=\frac{\text{Number of times term appears in document}}{\text{Total words in document}}$
TF(t)=Total words in documentNumber of times term appears in document

Example

Document:

"machine learning learning"

TF of learning:

$TF=2/3=0.67$
 $TF=\frac{2}{3}=0.67$
 $TF=3/2=0.67$

Inverse Document Frequency (IDF)

Measures:

rarity of word across documents

Why IDF needed?

Words appearing everywhere:

less important

Rare words:

more important

IDF Formula

$IDF(t)=\log(N/df)$
 $IDF(t)=\log\left(\frac{N}{df}\right)$
 $IDF(t)=\log(dfN)$

Where:

- N = total documents
 - df = documents containing word
-

Example

Suppose:

- total docs = 4
- learning appears in 2 docs

Then:

$$IDF = \log(4/2) = 0.693 \quad IDF = \log\left(\frac{4}{2}\right) = 0.693 \quad IDF = \log(2) = 0.693$$

TF-IDF Formula

MOST IMPORTANT FORMULA.

$$TF\text{-}IDF = TF \times IDF \quad TF\text{-}IDF = TF \times IDF \quad TF\text{-}IDF = TF \times IDF$$

Meaning

High TF-IDF means:

word is important for that document

TfidfVectorizer

```
vectorizer = TfidfVectorizer()
```

Automatically:

- tokenizes text
 - calculates TF
 - calculates IDF
 - creates matrix
-

TF-IDF Matrix

Rows:

documents

Columns:

terms/words

Values:

importance scores

Why transpose matrix?

`tfidf_df.T`

For better readability.

Top Terms

Code extracts:

top 3 most important words

per document.

Manual TF-IDF Calculation

Your code manually computes:

- TF
- IDF
- TF-IDF

for word:

learning

Logarithm in IDF

Why log used?

Without logarithm:

very large values

Log smooths scaling.

Applications of TF-IDF

Application	Example
Search engines	Google ranking
Chatbots	Important keyword extraction
Spam filtering	Spam word importance
Recommendation systems	Document similarity
NLP models	Feature extraction

Advantages of TF-IDF

Advantage	Explanation
Highlights important words	Rare but meaningful words get high score
Removes common word influence	Lowers stopword importance
Easy to compute	Efficient

Limitations of TF-IDF

Limitation	Explanation
Ignores word meaning	Only frequency-based
Ignores context	Same word different meanings
Sparse matrix	Large feature space

FULL ALGORITHM FOR EXAM WRITING

Algorithm: Text Preprocessing and TF-IDF using NLP

Step 1:

Import required libraries such as NLTK, Pandas, and Scikit-learn.

Step 2:

Download required NLTK resources like punkt, stopwords, wordnet, and POS tagger.

Step 3:

Define input text document.

Step 4:

Perform sentence tokenization using `sent_tokenize()`.

Step 5:

Perform word tokenization using `word_tokenize()`.

Step 6:

Apply POS tagging using `pos_tag()`.

Step 7:

Remove stopwords using stopword list.

Step 8:

Apply stemming using `PorterStemmer`.

Step 9:

Apply lemmatization using `WordNetLemmatizer`.

Step 10:

Create corpus of documents.

Step 11:

Apply TF-IDF using `TfidfVectorizer()`.

Step 12:

Generate TF-IDF matrix.

Step 13:

Find important terms using TF-IDF scores.

Step 14:

Interpret results.

MOST IMPORTANT VIVA QUESTIONS

Q1. What is NLP?

Branch of AI dealing with human language processing.

Q2. Difference between stemming and lemmatization?

Stemming	Lemmatization
Rule-based	Dictionary-based
Faster	More accurate
May produce invalid words	Produces meaningful words

Q3. Why stopwords removed?

They carry little semantic meaning.

Q4. What is TF-IDF?

Technique to measure word importance.

Q5. Why IDF used?

To reduce importance of common words.

Q6. Why logarithm used in IDF?

To normalize large values.

Q7. Difference between word tokenization and sentence tokenization?

Sentence Tokenization

Splits into sentences

Word Tokenization

Splits into words

Q8. What is corpus?

Collection of documents.

ONE-MINUTE VIVA EXPLANATION

“This practical performs NLP preprocessing and TF-IDF feature extraction. The document is first tokenized into sentences and words. POS tagging identifies grammatical roles. Stopwords are removed to reduce noise. Stemming and lemmatization reduce words to root forms. Then TF-IDF is applied to convert text into numerical vectors by measuring term importance using Term Frequency and Inverse Document Frequency. The TF-IDF matrix helps identify the most important words in each document.”

NLP Practical — Every Important Term Explained in Detail

This practical covers:

- NLP preprocessing
- Tokenization
- POS tagging
- Stopword removal
- Stemming
- Lemmatization
- TF-IDF

using:

- NLTK
 - Scikit-learn
-

1. NLP (Natural Language Processing)

Definition

NLP is a branch of:

Artificial Intelligence (AI)

that helps computers:

- understand
- analyze
- process
- generate

human language.

Why NLP Needed?

Humans communicate using:

- English
- Hindi
- Japanese
- text
- speech

Computers understand:

numbers and patterns

NLP converts:

human language → machine understandable format

Examples of NLP

Application	Example
Chatbots	ChatGPT
Translation	Google Translate
Voice Assistants	Siri, Alexa
Spam Detection	Gmail
Sentiment Analysis	Movie reviews

2. Text Preprocessing

Definition

Process of:

cleaning and preparing text

before applying machine learning.

Why preprocessing needed?

Raw text contains:

- punctuation
- stopwords
- noise
- inconsistent formats

Preprocessing improves:

- accuracy
 - speed
 - efficiency
-

Example

Before preprocessing:

"The cats are running quickly!"

After preprocessing:

["cat", "run", "quickly"]

3. NLTK

Definition

NLTK =

Natural Language Toolkit

A Python library used for:

- text processing
 - NLP operations
 - linguistic analysis
-

Uses of NLTK

Task	Example
Tokenization	Splitting words
POS Tagging	Grammar identification
Stopword Removal	Removing common words
Stemming	Root word extraction
Lemmatization	Dictionary root conversion

4. Tokenization

MOST IMPORTANT.

Definition

Process of breaking text into:

smaller units called tokens

Types of Tokenization

Type	Splits Into
Sentence Tokenization	Sentences
Word Tokenization	Words

Why tokenization important?

Machine learning models cannot process:

large paragraphs directly

They process:

tokens

5. Sentence Tokenization

Definition

Splitting paragraph into:

individual sentences

Example

Input:

"I love NLP. AI is interesting."

Output:

```
[  
"I love NLP.",  
"AI is interesting."  
]
```

Function Used

sent_tokenize()

Applications

- Text summarization
 - Chatbots
 - Translation systems
-

6. Word Tokenization

Definition

Splitting sentence into:

words/tokens

Example

Input:

"I love NLP"

Output:

["I", "love", "NLP"]

Function Used

`word_tokenize()`

Why important?

ML models use:

individual words as features

7. Token

Definition

Smallest meaningful unit of text.

Examples:

- word
 - punctuation
 - number
-

Example

Sentence:

"AI is amazing!"

Tokens:

["AI", "is", "amazing", "!"]

8. Corpus

Definition

Collection of documents/texts.

Example

```
corpus = [  
    "Document 1",  
    "Document 2"  
]
```

Why corpus important?

Machine learning learns patterns from:

multiple documents

9. Document

Definition

Single text inside corpus.

Example

Corpus:

```
[  
"I love NLP",  
"Machine learning is powerful"  
]
```

Each sentence:

individual document

10. POS Tagging

MOST IMPORTANT THEORY.

POS = Part Of Speech

Identifies grammatical role of words.

Why POS tagging needed?

Helps machine understand:

- grammar
 - sentence structure
 - meaning
-

Common POS Tags

Tag	Meaning
NN	Noun
NNP	Proper Noun
VB	Verb
VBG	Verb (gerund)
JJ	Adjective
RB	Adverb
DT	Determiner
IN	Preposition

Example

Sentence:

"Dogs bark loudly"

POS Tags:

Word	Tag
Dogs	NN
bark	VB
loudly	RB

Function Used

pos_tag()

Applications

- Grammar checking
- Translation
- Chatbots
- Question answering

11. Stopwords

VERY IMPORTANT.

Definition

Very common words that carry:

little semantic meaning

Examples

Stopword
s

the

is

are

and

of

Why remove stopwords?

Because they:

- increase noise
 - increase dimensionality
 - usually do not help prediction
-

Example

Before:

"The cat is on the mat"

After:

["cat","mat"]

Function Used

stopwords.words("english")

Applications

- Search engines
 - Spam filtering
 - Document analysis
-

12. isalpha()

Definition

Checks whether token contains:

only alphabetic characters

Why used?

Removes:

- punctuation
 - numbers
 - symbols
-

Example

Token isalpha()

NLP	True
123	False
AI!	False

13. Stemming

MOST IMPORTANT.

Definition

Process of reducing words into:

root/stem form

using rule-based chopping.

Example

Original	Stemmed
----------	---------

playing	play
---------	------

studies	studi
---------	-------

running	run
---------	-----

Function Used

PorterStemmer()

PorterStemmer

Popular stemming algorithm.

Why stemming used?

Reduces:

- vocabulary size
 - redundant forms
-

Problem with stemming

May produce:

invalid words

Example:

studies → studi

Applications

- Search engines
 - Information retrieval
 - Text classification
-

14. Lemmatization

VERY IMPORTANT.

Definition

Process of converting word into:

meaningful dictionary root word

Example

Original	Lemmatized
studies	study
running	run
better	good

Function Used

WordNetLemmatizer()

Difference Between Stemming and Lemmatization

Stemming	Lemmatization
Rule-based	Dictionary-based
Faster	More accurate
May create invalid words	Produces meaningful words

Why lemmatization better?

Maintains:

- proper meaning
 - grammar correctness
-

15. WordNet

Definition

A large English lexical database.

Contains:

- meanings
- synonyms

- root forms

Used by:

WordNetLemmatizer

16. TF (Term Frequency)

MOST IMPORTANT.

Definition

Measures:

how frequently a word appears in a document

Formula

$TF(t) = \frac{\text{Number of occurrences of term}}{\text{Total words in document}}$
 $TF(t) = \frac{\text{Number of occurrences of term}}{\text{Total words in document}}$

Example

Document:

"machine learning learning"

TF of learning:

$TF = \frac{2}{3} = 0.67$

Why TF important?

Higher TF means:

word important in document

17. IDF (Inverse Document Frequency)

VERY IMPORTANT.

Definition

Measures:

rarity of word across documents

Why IDF needed?

Common words:

less important

Rare words:

more important

Formula

$$IDF(t) = \log\left(\frac{N}{df}\right) \quad IDF(t) = \log(dfN)$$

Where:

- N = total documents
 - df = number of documents containing term
-

Example

Suppose:

- total docs = 4
- word appears in 2 docs

Then:

$$IDF = \log(4/2) = 0.693 \quad IDF = \log\left(\frac{4}{2}\right) = 0.693 \quad IDF = \log(2) = 0.693$$

Why logarithm used?

Without log:

very large values

Log normalizes scores.

18. TF-IDF

MOST IMPORTANT THEORY.

Full Form

Term Frequency – Inverse Document Frequency

Definition

Measures:

importance of word in document relative to corpus

Formula

$TF\text{-}IDF = TF \times IDF$

Meaning

High TF-IDF:

rare and important word

Low TF-IDF:

common unimportant word

Example

Word	TF-IDF
intelligence	High
the	Low

19. TfidfVectorizer

Definition

Scikit-learn tool that automatically:

- tokenizes
 - computes TF
 - computes IDF
 - creates vectors
-

Function Used

TfidfVectorizer()

Output

Creates:

TF-IDF Matrix

20. TF-IDF Matrix

Rows:

documents

Columns:

terms/words

Values:

importance scores

Example

	machine	learning
Doc1	0.2	0.8
Doc2	0.7	0.1

21. Feature Extraction

Definition

Converting text into:

numerical representation

for machine learning.

Why needed?

ML algorithms work on:

numbers, not raw text

22. Vectorization

Definition

Converting words/documents into:

numeric vectors

Example

["AI", "ML"]

becomes:

[0.5, 0.8]

23. Sparse Matrix

Definition

Matrix containing:

mostly zero values

Why TF-IDF creates sparse matrix?

Each document contains:

only few words

out of all possible vocabulary.

24. Vocabulary

Definition

Set of unique words in corpus.

Example

Corpus:

["AI is good", "AI uses data"]

Vocabulary:

["AI", "is", "good", "uses", "data"]

25. Dimensionality

Definition

Number of features/columns.

In TF-IDF:

number of unique words

High Dimensionality Problem

Too many words:

- increases memory
 - slows training
 - causes sparsity
-

26. Real-World Applications

Application	Example
Search Engines	Google

Spam Detection	Gmail
Chatbots	ChatGPT
Sentiment Analysis	Reviews
Recommendation Systems	Netflix
Document Classification	News categorization

27. MOST IMPORTANT EXTERNAL QUESTIONS

Q1. Why stopwords removed?

Because they carry little semantic meaning.

Q2. Difference between stemming and lemmatization?

Stemming	Lemmatization
Faster	More accurate
Rule-based	Dictionary-based
May produce invalid words	Produces valid words

Q3. Why TF-IDF used?

To identify important words.

Q4. Why IDF important?

Reduces importance of common words.

Q5. Why logarithm used in IDF?

To normalize large values.

Q6. What is tokenization?

Splitting text into tokens.

Q7. What is corpus?

Collection of documents.

Q8. What is vectorization?

Converting text into numerical vectors.

28. ONE-MINUTE FULL VIVA EXPLANATION

“This practical performs Natural Language Processing preprocessing and TF-IDF feature extraction. The text is tokenized into sentences and words. POS tagging identifies grammatical roles. Stopwords are removed to reduce noise. Stemming and lemmatization reduce words into root forms. TF-IDF is then applied to measure the importance of words in documents using Term Frequency and Inverse Document Frequency. Finally, the text is converted into numerical vectors for machine learning applications.”

Full Explanation — What Every Step Does

PART 1 — NLP Preprocessing Pipeline

Step 1 — Sentence Tokenization

python

```
sent_tokenize(document)
```

Splits the paragraph into **individual sentences** using punctuation like . ? !.

"Natural Language Processing is a fascinating field..." → 3 separate sentences

Step 2 — Word Tokenization


```
python
word_tokenize(document)
```

Splits each sentence into **individual words and punctuation tokens**.

"learning to understand" → ['learning', 'to', 'understand']

📌 Step 3 — POS Tagging

```
python
pos_tag(word_tokens)
```

Labels every word with its **grammatical role**:

Tag	Meaning	Example
NN	Noun	language, machine
VBG	Verb (gerund)	learning, applying
JJ	Adjective	fascinating
IN	Preposition	of, through
DT	Determiner	the, a
NNP	Proper Noun	Intelligence

📌 Step 4 — Stopword Removal

```
python
stopwords.words("english")
```

Removes **common words** that carry no meaning: *is, a, the, to, are, of, by* etc.
Keeps only meaningful content words for analysis.

📌 Step 5 — Stemming

```
python
PorterStemmer().stem(word)
```

Cuts words down to their **root form** by chopping suffixes — may not always be a real word:

learning → *learn* | *fascinating* → *fascin* | *applying* → *appli*

Fast but **rough** — doesn't care about real dictionary words.

📌 Step 6 — Lemmatization

```
python
WordNetLemmatizer().lemmatize(word)
```

Converts word to its **base dictionary form** — always a real word:

learning → learning | techniques → technique | models → model

Slower than stemming but **more accurate and meaningful**.

Stemming vs Lemmatization — Key Difference

Feature	Stemming	Lemmatization
Output	May not be real word	Always real word
Speed	Faster	Slower
Approach	Chops suffix	Uses dictionary
Example	studies → studi	studies → study

PART 2 — TF-IDF

What is TF-IDF?

A numerical score that tells **how important a word is in a document relative to all documents** in the corpus.

Formula:

TF (Term Frequency) = count of word in doc / total words in doc
IDF (Inverse Document Freq) = $\log(\text{Total docs} / \text{docs containing the word})$
TF-IDF = TF × IDF

Why IDF?

Words like "is", "the" appear in ALL documents → IDF becomes $\log(4/4) = 0$ → **TF-IDF = 0** → automatically filters out common words.

Words unique to one document → high IDF → **higher TF-IDF** → more important.

How to Present to Examiner

PART 1:

"I took a 3-sentence document about NLP. First I applied sentence tokenization to split it into sentences, then word tokenization to get individual words. POS tagging labeled each word's grammatical role. Then I removed stopwords — words like 'is', 'a', 'the' that carry no useful meaning. I applied PorterStemmer for stemming which aggressively trims word endings, and WordNetLemmatizer for lemmatization which gives proper base dictionary words. Lemmatization is preferred over stemming when accuracy matters."

PART 2:

"For TF-IDF, I created a corpus of 4 documents and used sklearn's TfidfVectorizer to build the TF-IDF matrix. Each row is a document, each column is a word, and each value is the TF-IDF score. A higher score means that word is more important and unique to that document. I also showed a manual calculation for the word 'learning' to prove the formula — $TF \times IDF$."

 If examiner asks "why use TF-IDF over just word count?"

"Word count gives high weight to common words like 'the' and 'is' which appear everywhere but carry no meaning. TF-IDF penalizes such words through IDF — words appearing in all documents get a score of 0, while unique meaningful words get a high score."

 If examiner asks "what is the difference between NLTK and sklearn TF-IDF?"

"NLTK is used for linguistic preprocessing like tokenization, POS tagging, stemming. Sklearn's TfidfVectorizer does the mathematical representation — it converts the cleaned text into a numeric matrix that machine learning models can use."

A8: Data Visualizatioon

Data Visualization Practical — Titanic Dataset



LINE-BY-LINE CODE EXPLANATION

This practical focuses on:

- Data Visualization
- Dataset Creation using NumPy
- Handling Missing Values
- Histogram
- Count Plot
- Scatter Plot
- Exploratory Data Analysis (EDA)

using:

- Pandas
- NumPy
- Matplotlib
- Seaborn



BLOCK 1 — Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Line	What it does	Why we need it
<code>import pandas as pd</code>	Loads Pandas library	Used for DataFrame creation and CSV handling
<code>import numpy as np</code>	Loads NumPy library	Used for random dataset generation and numerical operations
<code>import matplotlib.pyplot as plt</code>	Loads Matplotlib plotting module	Used for creating graphs and charts
<code>import seaborn as sns</code>	Loads Seaborn visualization library	Used for advanced statistical plots

Pandas

Definition

Pandas is a Python library used for:

- Data manipulation
 - Data cleaning
 - DataFrames
 - CSV handling
-

NumPy

Definition

NumPy =

Numerical Python

Used for:

- Arrays
 - Random data generation
 - Mathematical operations
-

Matplotlib

Definition

Basic plotting library in Python.

Used for:

- graphs
 - histograms
 - scatter plots
 - line plots
-

Seaborn

Definition

Advanced visualization library built on Matplotlib.

Provides:

- attractive plots
 - statistical visualization
 - easier syntax
-

BLOCK 2 — Create Titanic-Like Dataset

```
np.random.seed(42)  
n = 500
```

np.random.seed(42)

Definition

Fixes randomness.

Why used?

Without seed:

different dataset every run

With seed:

same dataset every run

Useful for:

- reproducibility
 - debugging
 - consistency
-

n = 500

Number of passengers/samples.

BLOCK 3 — Generate Categorical Data

```
sex = np.random.choice(["male", "female"], n, p=[0.65, 0.35])
```

np.random.choice()

Definition

Randomly selects values from given list.

Parameters

Parameter	Meaning
<code>["male", "female"]</code>	possible values
<code>n</code>	total values generated
<code>p=[0.65, 0.35]</code>	probabilities

Probability Distribution

65% male
35% female

Creates realistic Titanic-like distribution.

pclass


```
pclass = np.random.choice([1,2,3], n, p=[0.20,0.25,0.55])
```

Passenger class:

- 1 → First class
 - 2 → Second class
 - 3 → Third class
-

embarked

```
embarked = np.random.choice(["C","Q","S"], n)
```

Port where passenger boarded:

- C → Cherbourg
 - Q → Queenstown
 - S → Southampton
-

BLOCK 4 — Age Generation

```
age = np.where(  
    sex == "female",  
    np.random.normal(28, 12, n),  
    np.random.normal(30, 14, n)  
)
```

np.where()

Definition

Conditional selection function.

Meaning Here

If:

female → average age 28

male → average age 30

np.random.normal()

Definition

Generates values using:

Normal Distribution

Normal Distribution

Also called:

Gaussian Distribution

Bell-shaped distribution.

Most values near mean.

Parameters

Parameter	Meaning
28	mean
12	standard deviation
n	number of values

Mean

Average value.

Formula:

$$\text{Mean} = \frac{x_1 + x_2 + x_3 + \dots + x_n}{n}$$
$$\text{Mean} = \frac{x_1 + x_2 + x_3 + \dots + x_n}{n}$$

Standard Deviation

Measures spread of data.

High std:

data spread widely

Low std:

data clustered closely

clip(1,80)

`.clip(1,80)`

Restricts age between:

- minimum = 1
 - maximum = 80
-

round(1)

Rounds values to:

1 decimal place

BLOCK 5 — Survival Probability

```
surv_prob = np.where(sex == "female", 0.74, 0.19)
```

Meaning

Females given:

74% survival chance

Males:

19% survival chance

Based on Titanic history:

women survived more

Modify Survival Probability

```
surv_prob = np.where(pclass == 1,  
                     surv_prob + 0.15,  
                     surv_prob)
```

First class passengers:

higher survival chance

clip(0.05,0.95)

Ensures probability stays between:

5% and 95%

Binomial Distribution

```
survived = np.random.binomial(1, surv_prob)
```

np.random.binomial()

Used for:

binary outcomes

Possible values:

- 0
 - 1
-

Here

Value	Meaning
0	Not survived
1	Survived

BLOCK 6 — Fare Generation

```
fare = np.where(  
    pclass == 1,  
    np.random.normal(300,80,n),  
    ...  
)
```

Meaning

Fare depends on class.

Class	Average Fare
First	High
Second	Medium
Third	Low

Why realistic?

Higher-class passengers paid higher ticket fares.

BLOCK 7 — Create DataFrame

```
df = pd.DataFrame({...})
```

DataFrame

Definition

2-dimensional table structure in Pandas.

Like:

- Excel sheet

- SQL table
-

Columns Used

Column	Meaning
PassengerId	unique ID
Survived	target variable
Pclass	passenger class
Sex	gender
Age	age
Fare	ticket fare
Embarked	boarding port

BLOCK 8 — Missing Values

```
df.loc[missing_age, "Age"] = np.nan
```

Missing Values

Empty/unknown values.

Represented as:

NaN = Not a Number

Why added?

Real-world datasets often contain:

- incomplete forms
 - human errors
 - missing records
-

np.random.choice()

Randomly selects rows where values become missing.

BLOCK 9 — Save Dataset

```
df.to_csv("Titanic-Dataset.csv", index=False)
```

CSV

CSV =

Comma Separated Values

Stores tabular data in file form.

index=False

Prevents extra index column from being saved.

BLOCK 10 — Load Dataset

```
df = pd.read_csv("Titanic-Dataset.csv")
```

Loads CSV file into DataFrame.

BLOCK 11 — Display Dataset Info

```
df.shape
```

shape

Returns:

(rows, columns)

Example:

(500,7)

head()

Displays first 5 rows.

columns

Displays all column names.

BLOCK 12 — Check Missing Values

`df.isnull().sum()`

isnull()

Returns:

True for missing values

sum()

Counts total missing values.

BLOCK 13 — Handle Missing Values

`df["Age"] = df["Age"].fillna(df["Age"].median())`

fillna()

Fills missing values.

Why median for Age?

Age may contain outliers.

Median is robust to outliers.

Median

Middle value after sorting.

Mode

`mode()[0]`

Most frequent value.

Used for categorical columns.

Why mode for Embarked?

Because:

categorical text column

Mean/median impossible.

BLOCK 14 — Histogram

`sns.histplot(df["Fare"])`

Histogram

Definition

Graph showing:

frequency distribution

X-axis

Values/ranges.

Y-axis

Frequency/count.

bins=10

Divides data into:

10 intervals

kde=True

Adds:

Kernel Density Estimation curve

Smooth probability curve.

Right-Skewed Distribution

Most values small,
few extremely large values.

BLOCK 15 — Count Plot

```
sns.countplot(data=df,  
              x="Survived",  
              hue="Sex")
```

Count Plot

Shows:

frequency/count of categories

hue

Adds subgroup comparison.

Here:

male vs female

Purpose

Visualizes:

- survival count
 - gender comparison
-

BLOCK 16 — Scatter Plot

```
sns.scatterplot(  
    x="Age",  
    y="Fare",  
    hue="Survived"  
)
```

Scatter Plot

Shows relationship between:

two numerical variables

X-axis

Age

Y-axis

Fare

hue="Survived"

Different colors for:

- survived
 - not survived
-

alpha=0.7

Controls transparency.

Why scatter plot useful?

Helps detect:

- correlation
 - patterns
 - clusters
 - outliers
-

BLOCK 17 — savefig()

```
plt.savefig("plot.png")
```

savefig()

Saves graph image locally.

BLOCK 18 — Observations

```
print("""  
1. Fare distribution is right-skewed  
...  
""")
```

Why observations important?

Explains:

- trends
- insights
- patterns

from visualization.

Right-Skewed Distribution

Most values:

small

Few values:

very large

Tail extends right side.

The commented section creates:

manual Titanic dataset

instead of random generation.

Useful for:

- small datasets
- viva demonstrations
- controlled outputs

IMPORTANT TERMS USED

Term	Meaning
Data Visualization	Graphical representation of data
Histogram	Frequency distribution graph
Scatter Plot	Relationship between two variables
Count Plot	Category frequency graph
Missing Values	Empty/unknown values
Outliers	Extreme unusual values
Distribution	Pattern of data spread
Correlation	Relationship between variables
Gaussian Distribution	Bell-shaped distribution
KDE	Kernel Density Estimation
DataFrame	Table-like data structure

REAL-WORLD APPLICATIONS

Application	Example
Business Analytics	Customer trends
Healthcare	Disease statistics

Finance	Risk analysis
Social Media	User behavior
Transportation	Traffic analysis
Titanic Analysis	Survival analysis

FULL ALGORITHM FOR EXAM WRITING

Algorithm: Titanic Data Visualization using Python

Step 1:

Import required libraries such as Pandas, NumPy, Matplotlib, and Seaborn.

Step 2:

Generate Titanic-like dataset using NumPy random functions.

Step 3:

Create columns such as PassengerId, Sex, Age, Fare, Survived, and Embarked.

Step 4:

Insert missing values into selected columns.

Step 5:

Save dataset locally as CSV file.

Step 6:

Load dataset using Pandas.

Step 7:

Display dataset information and check missing values.

Step 8:

Handle missing values using median and mode imputation.

Step 9:

Create Histogram for Fare distribution.

Step 10:

Create Count Plot for Survival by Gender.

Step 11:

Create Scatter Plot for Age vs Fare.

Step 12:

Save plots and interpret observations.

MOST IMPORTANT VIVA QUESTIONS

Q1. Why use Seaborn?

For advanced statistical visualization.

Q2. Why histogram used?

To study frequency distribution.

Q3. Why scatter plot used?

To study relationship between variables.

Q4. Why median used for Age?

Robust against outliers.

Q5. Why mode used for Embarked?

Categorical column.

Q6. Why add missing values manually?

To simulate real-world datasets.

Q7. What is KDE?

Kernel Density Estimation curve.

Q8. Difference between histogram and countplot?

Histogram	Countplot
Numerical data	Categorical data
Distribution	Category frequency

ONE-MINUTE VIVA EXPLANATION

“This practical performs Titanic dataset visualization using Pandas, NumPy, Matplotlib, and Seaborn. A Titanic-like dataset is generated using NumPy random functions. Missing values are intentionally added and later handled using median and mode imputation. Histogram is used to visualize Fare distribution, Count Plot compares survival counts by gender, and Scatter Plot analyzes the relationship between Age and Fare. The graphs help identify patterns, distributions, and survival trends in the dataset.”

What 3 Plots Are in Each Practical

Practical	Histogram	Line Chart	Scatter Plot	Box Plot
P8	Fare distribution	Avg Fare by Class	Age vs Fare (survival)	—
P9	—	Survival Rate by Age Group	Age vs Fare (gender)	Age by Gender & Survival 
P10	All 4 features (2×2)	Mean per species	Petal L vs W	All 4 by species (2×2) 

What to Say to the Examiner

"For each practical, I have created three types of visualizations — a histogram or line chart to show distribution trends, a box plot to show spread and outliers, and a scatter plot to show relationships between two variables. Each plot is also saved as a PNG file automatically using `plt.savefig()` so it can be shown

even without a GUI. The `plt.show()` command opens an interactive window for each plot."

💡 **If examiner asks "why savefig()?"**

"On some Ubuntu machines, the display backend may not support interactive windows. `savefig()` ensures the plot is always saved to disk regardless, so we can always show the output."

A9: Data Visualization 2

Data Visualization Practical 9 — Titanic Dataset



LINE-BY-LINE CODE EXPLANATION

This practical focuses on:

- Box Plot
- Line Chart
- Scatter Plot
- Data Visualization
- Survival Analysis
- Exploratory Data Analysis (EDA)

using:

- NumPy
 - Pandas
 - Seaborn
 - Matplotlib
-



BLOCK 1 — Import Libraries

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

Library	Purpose
NumPy	Numerical operations and random dataset generation
Pandas	DataFrames and CSV handling
Seaborn	Statistical data visualization
Matplotlib	Plotting graphs and charts

NumPy

Definition

NumPy =

Numerical Python

Used for:

- arrays
 - random data generation
 - mathematical operations
-

Pandas

Used for:

- creating DataFrames
 - handling CSV files
 - data manipulation
-

Seaborn

Advanced visualization library built on:

Matplotlib

Provides:

- statistical plots
 - attractive themes
 - easy syntax
-

Matplotlib

Basic plotting library.

Used for:

- graphs
 - histograms
 - scatter plots
 - line charts
-

BLOCK 2 — Create Titanic-Like Dataset

```
np.random.seed(42)  
n = 500
```

np.random.seed(42)

Definition

Fixes randomness.

Why important?

Without seed:

different output every run

With seed:

same dataset every run

Useful for:

- reproducibility
 - debugging
 - consistent results
-

n = 500

Creates:

500 passengers

BLOCK 3 — Generate Gender Data

```
sex = np.random.choice(["male", "female"], n, p=[0.65, 0.35])
```

np.random.choice()

Definition

Randomly selects values from list.

Parameters

Parameter	Meaning
<code>["male", "female"]</code>	possible values
<code>n</code>	total samples
<code>p=[0.65, 0.35]</code>	probabilities

Probability Distribution

65% male
35% female

Creates realistic Titanic-like distribution.

BLOCK 4 — Passenger Class Generation

```
pclass = np.random.choice([1,2,3], n, p=[0.20,0.25,0.55])
```

Pclass

Passenger ticket class.

Value	Meaning
1	First Class
2	Second Class

Why probabilities used?

Most Titanic passengers were:

third class

BLOCK 5 — Age Generation

```
age = np.where(  
    sex == "female",  
    np.random.normal(28,12,n),  
    np.random.normal(30,14,n)  
)
```

np.where()

Definition

Conditional selection function.

Meaning

If:

female → average age 28
male → average age 30

np.random.normal()

Definition

Generates values using:

Normal (Gaussian) Distribution

Gaussian Distribution

Also called:

Bell-shaped distribution

Most values near mean.

Parameters

Parameter	Meaning
28	Mean
12	Standard deviation
n	number of values

Mean

Average value.

Formula:

$$\text{Mean} = \frac{x_1 + x_2 + x_3 + \dots + x_n}{n}$$
$$\text{Mean} = \frac{x_1 + x_2 + x_3 + \dots + x_n}{n}$$

Standard Deviation

Measures spread of data.

High std:

data widely spread

Low std:

data close together

clip(1,80)

`.clip(1,80)`

Restricts age between:

- minimum = 1
 - maximum = 80
-



BLOCK 6 — Survival Probability

```
surv_prob = np.where(sex == "female", 0.74, 0.19)
```

Meaning

Females:

74% survival chance

Males:

19% survival chance

Based on real Titanic survival patterns.

Modify Survival by Class

```
surv_prob = np.where(pclass == 1,  
                      surv_prob + 0.15,  
                      surv_prob)
```

First-class passengers:

higher survival probability

clip(0.05,0.95)

Keeps probability between:

5% and 95%

Binomial Distribution

```
survived = np.random.binomial(1, surv_prob)
```

np.random.binomial()

Definition

Used for:

binary outcomes

Possible outputs:

- 0
 - 1
-

Here

Value	Meaning
0	Not Survived
1	Survived

BLOCK 7 — Fare Generation

```
fare = np.where(  
    pclass == 1,  
    np.random.normal(80,40,n),  
    ...  
)
```

Meaning

Fare depends on passenger class.

Class	Average Fare
-------	-----------------

First	Highest
Second	Medium
Third	Lowest

Why realistic?

First-class passengers paid expensive ticket fares.

BLOCK 8 — Create DataFrame

```
df = pd.DataFrame({...})
```

DataFrame

Definition

2-dimensional table structure.

Similar to:

- Excel sheet
 - SQL table
-

Columns Used

Column	Meaning
sex	passenger gender
age	passenger age
survived	survival status
fare	ticket fare
pclass	passenger class

BLOCK 9 — Save Dataset

```
df.to_csv("Titanic_Practical9_Dataset.csv", index=False)
```

CSV

CSV =

Comma Separated Values

Stores tabular data in file form.

index=False

Prevents extra index column from saving.

BLOCK 10 — Dataset Information

```
df.shape
```

shape

Returns:

(rows, columns)

Example:

(500,5)

head()

Displays first rows.

value_counts()

Counts frequency of categories.

Example:

```
df["survived"].value_counts()
```

Counts:

- survived
 - not survived
-



PLOT 1 — BOX PLOT

```
sns.boxplot(data=df,  
            x="sex",  
            y="age",  
            hue="survived")
```

Box Plot

MOST IMPORTANT THEORY.

Definition

Graph used to visualize:

- distribution
- spread
- outliers

of numerical data.

What Box Plot Shows

Component	Meaning
Median line	middle value

Box	interquartile range
Whiskers	spread of data
Dots outside	outliers

Median

Middle value after sorting.

Interquartile Range (IQR)

Difference between:

- Q3 (75%)
- Q1 (25%)

Formula:

$$\text{IQR} = \text{Q}_3 - \text{Q}_1$$

Outliers

Extreme unusual values.

hue="survived"

Separates plot by:

- survived
 - not survived
-

palette

Changes colors of categories.

plt.legend()

Adds legend explanation.

Why box plot useful?

Helps detect:

- outliers
 - spread
 - skewness
 - group comparison
-



PLOT 2 — LINE CHART

```
df["age_group"] = pd.cut(df["age"], bins=[...])
```

pd.cut()

Definition

Converts continuous data into:

intervals/groups

Age Groups

Group	Range
0-10	children
11-20	teens
21-30	young adults

etc.

Why grouping needed?

Makes analysis easier.

groupby()

```
df.groupby("age_group")
```

groupby()

Groups rows based on category.

mean()

Calculates average.

Here:

average survival rate

reset_index()

Converts grouped result back into DataFrame.

Line Plot

```
sns.lineplot()
```

Definition

Shows:

trend/change over continuous categories

marker="o"

Adds circular markers.

linewidth=2.5

Controls line thickness.

Why line chart useful?

Helps identify:

- trends
 - progression
 - patterns over intervals
-

Survival Rate

Definition

Percentage/probability of passengers surviving.

Formula:

$$\text{Survival Rate} = \frac{\text{Number of Survivors}}{\text{Total Passengers}}$$
$$\text{Survival Rate} = \frac{\text{Number of Survivors}}{\text{Total Passengers}}$$



PLOT 3 — SCATTER PLOT

```
sns.scatterplot(  
    data=df,  
    x="age",  
    y="fare",
```

```
hue="sex",  
style="survived"  
)
```

Scatter Plot

Definition

Graph showing relationship between:

two numerical variables

X-axis

Age

Y-axis

Fare

hue="sex"

Different colors for:

- male
 - female
-

style="survived"

Different marker styles for:

- survived
 - not survived
-

alpha=0.6

Controls transparency.

Why scatter plot useful?

Helps identify:

- correlation
 - clusters
 - trends
 - outliers
-

Correlation

Definition

Relationship between variables.

Positive Correlation

Both increase together.

Negative Correlation

One increases, other decreases.

 **savefig()**

`plt.savefig("plot.png")`

savefig()

Saves plot image locally.

Observations

```
print("""
...
""")
```

Why observations important?

Explain:

- trends
- insights
- conclusions

from graphs.

IMPORTANT TERMS USED

Term	Meaning
Data Visualization	Graphical representation of data
Box Plot	Distribution visualization
Scatter Plot	Relationship graph
Line Chart	Trend visualization
Distribution	Data spread pattern
Outlier	Extreme value
Correlation	Relationship between variables
IQR	Interquartile range
Median	Middle value
Survival Rate	Percentage survived
Gaussian Distribution	Bell-shaped distribution
Transparency	Opacity level

REAL-WORLD APPLICATIONS

Application	Example
Healthcare	Patient survival analysis
Finance	Risk visualization
Marketing	Customer trend analysis
Transportation	Passenger behavior
Insurance	Risk assessment
Data Science	Exploratory Data Analysis

FULL ALGORITHM FOR EXAM WRITING

Algorithm: Titanic Data Visualization using Box Plot, Line Chart, and Scatter Plot

Step 1:

Import required libraries such as NumPy, Pandas, Seaborn, and Matplotlib.

Step 2:

Generate Titanic-like dataset using NumPy random functions.

Step 3:

Create columns such as sex, age, fare, pclass, and survived.

Step 4:

Save dataset locally as CSV file.

Step 5:

Display dataset information and class distributions.

Step 6:

Create Box Plot for Age distribution by Gender and Survival.

Step 7:

Create age groups using `pd.cut()`.

Step 8:

Calculate survival rate by age group using `groupby()` and `mean()`.

Step 9:

Create Line Chart for Survival Rate by Age Group.

Step 10:

Create Scatter Plot for Age vs Fare using Gender and Survival categories.

Step 11:

Save plots and interpret observations.

MOST IMPORTANT VIVA QUESTIONS

Q1. What is Box Plot?

Graph showing distribution and outliers.

Q2. What is IQR?

Difference between Q3 and Q1.

Q3. Why line chart used?

To show trends over intervals.

Q4. Why scatter plot used?

To analyze relationship between variables.

Q5. What is correlation?

Relationship between variables.

Q6. Why use `pd.cut()`?

To group continuous values into intervals.

Q7. What are outliers?

Extreme unusual values.

Q8. Difference between histogram and boxplot?

Histogram	Boxplot
Frequency distribution	Distribution summary
Uses bins	Uses quartiles

ONE-MINUTE VIVA EXPLANATION

“This practical performs Titanic dataset visualization using Box Plot, Line Chart, and Scatter Plot. A Titanic-like dataset is generated using NumPy random functions. Box Plot visualizes age distribution by gender and survival status, helping identify spread and outliers. Line Chart analyzes survival rate across age groups using grouped averages. Scatter Plot shows the relationship between Age and Fare while comparing Gender and Survival categories. These visualizations help understand survival trends, passenger behavior, and data distribution patterns.”

A10: Data Visualization 3

Practical 10 — Iris Dataset Visualization (Detailed Viva + Theory Notes)

This practical performs:

- Data Visualization
- Exploratory Data Analysis (EDA)
- Histogram
- Box Plot
- Line Chart
- Scatter Plot
- Outlier Detection

using:

- Iris Dataset from sklearn
 - Seaborn
 - Matplotlib
 - Pandas
-

MAIN GOAL OF THIS PRACTICAL

The objective is to:

- visualize Iris dataset features
 - compare flower species
 - analyze distributions
 - identify outliers
 - study relationships between features
-

WHAT IS IRIS DATASET?

The Iris dataset is one of the most famous datasets in Machine Learning.

Contains:

- 150 flower samples
 - 3 flower species
 - 4 numerical features
-

Species Present

Species

Setosa

Versicolor

Virginica

Features

Feature	Meaning
sepal_length	Length of sepal
sepal_width	Width of sepal
petal_length	Length of petal
petal_width	Width of petal

BLOCK 1 — Import Libraries

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

```
from sklearn.datasets import load_iris
```

seaborn

Definition

Advanced visualization library built on:

Matplotlib

Used for:

- statistical plots
- beautiful visualizations

- simplified plotting
-

matplotlib.pyplot

Basic plotting library.

Used for:

- graphs
 - charts
 - histograms
 - scatter plots
-

pandas

Used for:

- DataFrames
 - CSV handling
 - tabular data operations
-

numpy

Used for:

- numerical operations
 - arrays
 - calculations
-

load_iris

Loads Iris dataset from:

sklearn library

No internet required.

BLOCK 2 — Load Iris Dataset

```
iris = load_iris(as_frame=True)
```

as_frame=True

Loads dataset as:

Pandas DataFrame

instead of NumPy arrays.

DataFrame

Definition

2-dimensional table-like structure.

Similar to:

- Excel sheet
 - SQL table
-

BLOCK 3 — Create DataFrame

```
df = iris.frame
```

Stores dataset inside:

df DataFrame

BLOCK 4 — Map Species Names

```
df["species"] = df["target"].map({  
    0:"setosa",  
    1:"versicolor",
```

```
2:"virginica"  
})
```

map()

Definition

Replaces values using mapping dictionary.

Why needed?

Original target column contains:

0,1,2

Converted into:

species names

for readability.

BLOCK 5 — Drop Target Column

```
df.drop("target", axis=1, inplace=True)
```

drop()

Removes column or row.

axis=1

Means:

column operation

inplace=True

Directly modifies original DataFrame.

BLOCK 6 — Rename Columns

```
df.columns = [...]
```

Why rename columns?

Makes column names:

- shorter
 - readable
 - cleaner
-

BLOCK 7 — Save Dataset

```
df.to_csv("Iris_Dataset.csv", index=False)
```

CSV

CSV =

Comma Separated Values

Used for storing tabular data.

index=False

Prevents extra index column from saving.

BLOCK 8 — Dataset Information

`df.shape`

shape

Returns:

(rows, columns)

Example:

(150,5)

head()

Displays first rows.

BLOCK 9 — Feature Types

Numeric (continuous)

Continuous Variable

Numeric variable that can take:

any value in range

Examples:

- 5.1
- 6.7
- 3.2

Nominal Variable

Categorical variable with:

no ordering

Example:

- setosa
- virginica

PLOT 1 — HISTOGRAM

`sns.histplot()`

MOST IMPORTANT THEORY.

Histogram

Definition

Graph showing:

frequency distribution

of numerical data.

X-axis

Feature values.

Y-axis

Frequency/count.

bins=20

Divides data into:

20 intervals

called:

bins

KDE

kde=True

Adds:

Kernel Density Estimation curve

Smooth probability distribution curve.

Why histogram useful?

Helps identify:

- distribution
 - skewness
 - spread
 - frequency patterns
-

Distribution

Definition

Pattern showing how data values spread.

Normal Distribution

Bell-shaped distribution.

Most values near mean.

Skewness

Definition

Asymmetry in distribution.

Right Skew

Tail extends right side.

Left Skew

Tail extends left side.

`plt.subplots()`

`fig, axes = plt.subplots(2,2)`

Creates:

multiple plots in grid

`axes.flatten()`

Converts 2D axes into:

1D iterable list

zip()

Pairs:

- axes
- features
- colors

together.

PLOT 2 — BOX PLOT

`sns.boxplot()`

MOST IMPORTANT THEORY.

Box Plot

Definition

Graph showing:

- distribution
 - spread
 - quartiles
 - outliers
-

Components of Box Plot

Component	Meaning
Median line	middle value
Box	IQR

Whiskers	spread
Dots	outliers

Quartiles

Quartile	Meaning
Q1	25th percentile
Q2	median
Q3	75th percentile

Median

Middle value after sorting.

IQR (Interquartile Range)

Formula:

$IQR = Q_3 - Q_1$

Outliers

Extreme unusual values.

hue="species"

Creates separate colored categories for:

- setosa
 - versicolor
 - virginica
-

palette

Controls plot colors.

legend=False

Hides legend.

Why box plot useful?

Helps detect:

- outliers
 - spread
 - median comparison
 - variability
-



PLOT 3 — LINE CHART

`groupby("species")`

groupby()

Groups rows by category.

mean()

Calculates average.

Formula:

$$\text{Mean} = \frac{x_1 + x_2 + x_3 + \dots + x_n}{n}$$

melt()

`mean_df.melt()`

Converts:

wide format → long format

needed for Seaborn lineplot.

Wide Format

Features in separate columns.

Long Format

Single column for features and values.

Line Plot

`sns.lineplot()`

Definition

Shows:

trends/changes across categories

marker="o"

Adds circular markers.

linewidth

Controls line thickness.

Why line chart useful?

Helps compare:

- trends
- averages
- feature progression

between species.

PLOT 4 — SCATTER PLOT

`sns.scatterplot()`

MOST IMPORTANT THEORY.

Scatter Plot

Definition

Graph showing relationship between:

two numerical variables

X-axis

`petal_length`

Y-axis

petal_width

hue="species"

Different colors for:

- setosa
 - versicolor
 - virginica
-

s=80

Controls marker size.

alpha=0.8

Controls transparency.

Why scatter plot useful?

Helps detect:

- correlation
 - clusters
 - separation
 - trends
-

Correlation

Definition

Relationship between variables.

Positive Correlation

Both variables increase together.

Example:

petal length ↑

petal width ↑

Negative Correlation

One increases while other decreases.

Cluster

Group of similar data points.

Species Separation

Setosa appears clearly separated from other species.

OUTLIER DETECTION

quantile(0.25)

Quantile

Value below certain percentage of data.

Q1

25th percentile.

Q3

75th percentile.

IQR Method

Formula:

$$\text{IQR} = Q_3 - Q_1$$

Outlier Formula

$$\text{Lower Bound} = Q_1 - 1.5(\text{IQR})$$

$$\text{Upper Bound} = Q_3 + 1.5(\text{IQR})$$

Why $1.5 \times \text{IQR}$?

Standard statistical rule for detecting outliers.

Outliers

Values:

- unusually high
- unusually low

compared to rest of data.

Why outlier detection important?

Outliers may:

- distort analysis
 - affect ML models
 - indicate anomalies/errors
-

OBSERVATIONS

Histogram Observation

Petal features clearly separate species.

Box Plot Observation

Setosa has:

smallest petals

Virginica:

largest petals

Line Chart Observation

Virginica shows:

highest average measurements

Scatter Plot Observation

Setosa forms:

distinct cluster

easy to classify.

IMPORTANT TERMS USED

Term	Meaning
Data Visualization	Graphical representation of data
Histogram	Frequency distribution graph
Box Plot	Distribution and outlier graph
Scatter Plot	Relationship graph
Line Chart	Trend graph
Distribution	Data spread pattern
Outlier	Extreme value
Quartile	Percentile division
IQR	Interquartile Range
Correlation	Relationship between variables
Cluster	Group of similar points
KDE	Kernel Density Estimation

REAL-WORLD APPLICATIONS

Application	Example
Biology	Species analysis
Healthcare	Patient measurements
Finance	Stock trend analysis
Marketing	Customer segmentation
Machine Learning	Feature analysis
Data Science	Exploratory Data Analysis

FULL ALGORITHM FOR EXAM WRITING

Algorithm: Iris Dataset Visualization using Histogram, Box Plot, Line Chart, and Scatter Plot

Step 1:

Import required libraries such as Pandas, NumPy, Seaborn, Matplotlib, and sklearn.

Step 2:

Load Iris dataset using `load_iris()` from sklearn.

Step 3:

Create DataFrame and add species names.

Step 4:

Remove target column and rename feature columns.

Step 5:

Save dataset locally as CSV file.

Step 6:

Display dataset information and feature types.

Step 7:

Create Histograms for all numerical features.

Step 8:

Create Box Plots to compare feature distributions by species.

Step 9:

Calculate mean feature values using `groupby()`.

Step 10:

Create Line Chart for average feature comparison.

Step 11:

Create Scatter Plot for petal length vs petal width.

Step 12:

Detect outliers using IQR method.

Step 13:

Interpret observations and analyze species patterns.

MOST IMPORTANT VIVA QUESTIONS

Q1. Why histogram used?

To study frequency distribution.

Q2. What is KDE?

Kernel Density Estimation curve.

Q3. What is Box Plot?

Graph showing distribution and outliers.

Q4. What is IQR?

Difference between Q3 and Q1.

Q5. Why scatter plot used?

To study relationship between variables.

Q6. What is correlation?

Relationship between variables.

Q7. Why Setosa easily separable?

Petal features differ significantly.

Q8. Why use IQR method?

To statistically detect outliers.

ONE-MINUTE VIVA EXPLANATION

“This practical performs visualization and exploratory analysis on the Iris dataset using Histograms, Box Plots, Line Charts, and Scatter Plots. Histograms analyze feature distributions, Box Plots compare distributions and detect outliers, Line Charts compare mean feature values across species, and Scatter Plots study relationships between petal features. Outliers are detected using the IQR method. These visualizations help understand species separation, feature variability, and data patterns in the Iris dataset.”

A11: Hadoop

Hadoop Practical — MapReduce WordCount

This practical implements:

Hadoop MapReduce WordCount Program

using:

- Java
- Hadoop Framework
- MapReduce Programming Model

Goal:

Count how many times each word appears in a file

Example:

Input:

Hello World Hello Hadoop

Output:

Hello 2

World 1

Hadoop 1



WHAT IS HADOOP?

MOST IMPORTANT THEORY QUESTION.

Definition

Hadoop is:

Big Data Framework

used to:

- store huge data
- process huge data
- analyze huge data

across multiple computers.

Why Hadoop Needed?

Normal computers struggle with:

- TBs of data
- PBs of data
- millions of files

Hadoop solves this by:

splitting work across many computers

Real-Life Example

Suppose:

One child has to count all books in huge library

Very slow.

Instead:

100 children count different shelves

Very fast.

That is Hadoop.

Hadoop Main Components

Component	Work
HDFS	Storage
MapReduce	Processing
YARN	Resource management

1. HDFS

HDFS =

Hadoop Distributed File System

Definition

Stores huge files across multiple systems.

Example

Instead of storing:

1 huge file in one computer

HDFS stores:

small pieces in many computers

Why Distributed Storage?

Advantages:

- faster access
- fault tolerance
- scalability

Fault Tolerance

Definition

System continues working even if:
one computer fails

Scalability

Definition

Ability to:
add more computers easily

2. MapReduce

MOST IMPORTANT THEORY.

Definition

Programming model used for:

parallel data processing

in Hadoop.

Two Main Phases

Phase

Work

Map	Break and process data
Reduce	Combine final results

e

Simple Example

Input:

Hello Hadoop Hello

Mapper Output

(Hello,1)
(Hadoop,1)
(Hello,1)

Reducer Output

Hello → 2
Hadoop → 1

Child-Friendly Understanding

Imagine:

Teacher gives chocolates counting work to students

Mapper

Each student counts:

small group chocolates

Reducer

Teacher combines:

all students' counts

WORDCOUNT PROGRAM

MOST FAMOUS HADOOP PROGRAM.

Goal

Count frequency of words in text file.

Example

Input:

Cat Dog Cat

Output:

Cat 2

Dog 1

JAVA IMPORTS

```
import java.io.IOException;
```

IOException

Exception caused during:

- file handling
- reading/writing files

Exception

Definition

Unexpected error during execution.

`import java.util.*`

Used for utility classes.

StringTokenizer

Used for:

splitting sentence into words

Hadoop Imports

```
import org.apache.hadoop.fs.Path;
```

Path

Represents:

file/folder path

Configuration

```
import org.apache.hadoop.conf.*;
```


Stores Hadoop configuration settings.

Writable Classes

```
import org.apache.hadoop.io.*;
```

Why Writable?

Hadoop transfers data between machines.

Needs:

special serializable data types

Serialization

Definition

Converting object into transferable format.

Hadoop Data Types

Hadoop Type	Java Equivalent
-------------	-----------------

IntWritable	int
-------------	-----

LongWritable	long
--------------	------

Text	String
------	--------

Why Hadoop uses Writable?

Normal Java types:

not optimized for distributed transfer

Mapper Class

public static class Map
extends Mapper<LongWritable, Text, Text, IntWritable>

MOST IMPORTANT.

Mapper

Definition

Processes input data line by line.

Generic Parameters

Mapper<InputKey, InputValue, OutputKey, OutputValue>

Here

Parameter	Meaning
LongWritable	line position
Text	line text
Text	output word
IntWritable	output count

LongWritable Key

Represents:

byte position of line in file

Usually ignored.

Text Value

Represents:

one line from file

Mapper Output

Produces:

(word,1)

pairs.

IntWritable one = new IntWritable(1)

Stores:

constant value 1

Why always 1?

Each word occurrence counts as:

one appearance

Text word = new Text()

Stores current word token.

map() Function

public void map(...)

Definition

Core function executed for every input line.

value.toString()

Converts Hadoop Text into:

normal Java String

StringTokenizer

```
StringTokenizer tokenizer = new StringTokenizer(line);
```

Splits sentence into:

words/tokens

Example

Input:

Hello Hadoop World

Tokens:

Hello
Hadoop
World

tokenizer.hasMoreTokens()

Checks:

more words remaining?

tokenizer.nextToken()

Returns:

next word

word.set()

Stores current token into:

Text object

context.write(word, one)

MOST IMPORTANT.

Outputs:

(word,1)

pair.

Example

Input:

Hello Hadoop Hello

Mapper output:

(Hello,1)

(Hadoop,1)

(Hello,1)

Reducer Class

public static class Reduce
extends Reducer<Text, IntWritable, Text, IntWritable>

MOST IMPORTANT.

Reducer

Definition

Combines all mapper outputs.

Reducer Input

(word,[1,1,1,1])

All counts grouped together.

Example

("Hello",[1,1])

reduce() Function

Processes grouped data.

sum variable

Stores:

total count

for loop

`for(IntWritable val : values)`

Iterates through:

all 1s

`val.get()`

Extracts integer value from:

`IntWritable`

`sum += val.get()`

Adds counts together.

Example

`1 + 1 + 1 = 3`

`context.write(key,new IntWritable(sum))`

Outputs final result.

Example:

Hello 3

MAIN FUNCTION

`public static void main(String[] args)`

Program starting point.

Configuration conf

Stores Hadoop settings.

Job

```
Job job = new Job(conf,"wordcount");
```

Creates Hadoop job.

Job Name

wordcount

used for identification.

setOutputKeyClass()

Defines output key datatype.

Here:

Text

setOutputValueClass()

Defines output value datatype.

Here:

IntWritable

setMapperClass()

Specifies Mapper class.

setReducerClass()

Specifies Reducer class.

InputFormat

TextInputFormat

Reads file:

line by line

OutputFormat

TextOutputFormat

Writes:

plain text output

FileInputFormat.addInputPath()

Sets input folder path.

FileOutputFormat.setOutputPath()

Sets output folder path.

args[0]

Input path from terminal.

args[1]

Output path from terminal.

job.waitForCompletion(true)

Runs job and waits until completion.



TERMINAL COMMANDS

MOST IMPORTANT FOR EXTERNAL.

STEP 1 — Create Input Folder

```
mkdir -p ~/wordcount/input
```

mkdir

Creates directory/folder.

-p

Creates parent folders if needed.

echo Command

```
echo "Hello Hadoop" > file.txt
```

Writes text into file.

STEP 2 — Compile Java Program

```
hadoop com.sun.tools.javac.Main WordCount.java
```

Compilation

Converts:

.java → .class

bytecode.

STEP 3 — Create JAR File

```
jar cf WordCount.jar WordCount*.class
```

JAR

JAR =

Java ARchive

Package containing compiled classes.

STEP 4 — Run Hadoop Job

```
hadoop jar WordCount.jar WordCount input output
```

Meaning

Part	Meaning
hadoop jar	run Hadoop job
WordCount.jar	jar file
WordCount	main class
input	input folder
output	output folder

STEP 5 — View Output

`hadoop fs -cat output/part-r-00000`

hadoop fs

Hadoop file system command.

cat

Displays file contents.

part-r-00000

Reducer output file.

Why output folder deletion needed?

Hadoop does NOT overwrite existing output folders.

Must delete first.

rm -rf

Deletes folder recursively.

IMPORTANT TERMS USED

Term	Meaning
Big Data	Extremely huge data
Hadoop	Big data processing framework
HDFS	Distributed storage system
MapReduce	Parallel processing model
Mapper	Breaks/processes data
Reducer	Combines results
Serialization	Object conversion for transfer
Tokenization	Splitting text into words
Writable	Hadoop data types
Job	Hadoop task
JAR	Java archive file

MAPREDUCE FLOW

INPUT

Hello Hadoop Hello

MAPPER OUTPUT

(Hello,1)
(Hadoop,1)

(Hello,1)

SHUFFLE & SORT

Groups same words.

Hello → [1,1]

Hadoop → [1]

REDUCER OUTPUT

Hello 2

Hadoop 1

REAL-WORLD APPLICATIONS

Application	Example
Search Engines	Google indexing
Social Media	Tweet analysis
Banking	Fraud detection
E-commerce	Recommendation systems
Healthcare	Patient data analysis
Log Analysis	Server logs

FULL ALGORITHM FOR EXAM WRITING

Algorithm: Hadoop MapReduce WordCount

Step 1:

Create input file containing text data.

Step 2:

Write Mapper class to split lines into words.

Step 3:

Emit (`word, 1`) pair for every word.

Step 4:

Hadoop framework groups same words together.

Step 5:

Reducer class receives grouped words.

Step 6:

Sum all counts for each word.

Step 7:

Emit final (`word, totalCount`) pair.

Step 8:

Compile Java program.

Step 9:

Create JAR file.

Step 10:

Run Hadoop MapReduce job.

Step 11:

Display final output.

MOST IMPORTANT VIVA QUESTIONS

Q1. What is Hadoop?

Framework for distributed big data processing.

Q2. What is HDFS?

Distributed storage system of Hadoop.

Q3. What is MapReduce?

Parallel processing programming model.

Q4. What is Mapper?

Processes input and emits key-value pairs.

Q5. What is Reducer?

Combines mapper outputs.

Q6. Why IntWritable used?

Hadoop optimized serializable integer datatype.

Q7. What is tokenization?

Splitting text into words.

Q8. Why output folder deleted before rerun?

Hadoop cannot overwrite existing output directory.

ONE-MINUTE VIVA EXPLANATION

“This practical implements Hadoop MapReduce WordCount using Java. Hadoop is a big data framework that stores and processes huge datasets in distributed systems. The Mapper reads input text line by line, tokenizes words, and emits `(word, 1)` pairs. Hadoop groups identical words together during shuffle and

sort phase. The Reducer sums all counts and produces final word frequencies. The program is compiled into a JAR file and executed using Hadoop commands in Ubuntu terminal.”

DataSets Guide

Scikit-learn Built-in Datasets Guide



DATASETS IN YOUR SYLLABUS & HOW TO LOAD THEM

◆ 1. IRIS Dataset ✓ (Fully Built-in)

```
from sklearn.datasets import load_iris
import pandas as pd

# Load
iris = load_iris()

# Convert to DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target
df["species"] = df["target"].map({0: "setosa", 1: "versicolor", 2: "virginica"})

print(df.head())
print(df.shape)
print(df.describe())
```

◆ 2. TITANIC Dataset ✗ (NOT built-in)

```
# Titanic is NOT in sklearn — must use CSV
df = pd.read_csv("Titanic-Dataset.csv")

# ⚠ EXAM SAFETY CODE — handle if file not found
import os
print(os.listdir()) # See what files are available
```

Titanic is available via **seaborn** though as a backup:

```
# BACKUP if no CSV file is present
import seaborn as sns
df = sns.load_dataset("titanic")
print(df.head())
```

◆ 3. BOSTON HOUSING Dataset ⚠ (Removed — use California instead)

```
# ✗ REMOVED from sklearn in version 1.2+ due to ethical issues
# from sklearn.datasets import load_boston ← THIS WILL ERROR
```

```
# ✓ CORRECT REPLACEMENT — California Housing
from sklearn.datasets import fetch_california_housing
import pandas as pd
```

```
housing = fetch_california_housing()
df = pd.DataFrame(housing.data, columns=housing.feature_names)
df["Price"] = housing.target
```

```
print(df.head())
print(df.shape)
```

⚠️ **Tell examiner:** "Boston dataset was deprecated in sklearn 1.2 due to ethical concerns about a racial variable. California Housing is the standard replacement."

UNIVERSAL EXAM SAFETY CODE

Paste this at the start of ANY practical if you're unsure:

```
import os
import pandas as pd

# — See all files in current directory —
print("Files available:", os.listdir())

# — Try loading CSV, fall back to built-in —
try:
    df = pd.read_csv("Titanic-Dataset.csv") # change name as needed
    print("✅ CSV loaded successfully")
except FileNotFoundError:
    print("⚠️ CSV not found, loading from seaborn...")
    import seaborn as sns
    df = sns.load_dataset("titanic")
    print("✅ Seaborn Titanic loaded")

print(df.head())
```

KEY THING TO REMEMBER FOR EXAM

For IRIS — always safe to write both options:

Option 1: From CSV
df = pd.read_csv("iris.csv")

Option 2: From sklearn (no file needed — RECOMMENDED)
from sklearn.datasets import load_iris
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target

To show the dataset of the Dataframe

show_df_summary(df, "Student Performance")

Just call `show_df_summary(df)` whenever examiner says **"show me your DataFrame"**